A CLOUD-BASED TRAFFIC CONTROL SYSTEM FOR SUB-SAHARAN AFRICAN CITIES COMBINING CHAIN-OF-THOUGHT, VISION-LANGUAGE-ACTION MODELS AND LATENT WORLD MODELS

Cloud-Hosted Reasoning with Edge Fallback for Intermittent Connectivity

BY

ARIYO, TEMITOPE WILLIAMS

249074175

SUBMITTED TO

THE DEPARTMENT OF COMPUTER SCIENCES, FACULTY OF COMPUTING AND INFORMATICS, SCHOOL OF POSTGRADUATE STUDIES, UNIVERSITY OF LAGOS, LAGOS, NIGERIA.

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF

MASTER OF INFORMATION TECHNOLOGY IN COMPUTER SCIENCE

OCTOBER, 2026

CERTIFICATION

This is to certify that Ariyo, Temitope Williams with Matriculation Number 249074175 carried out this project entitled "**A CLOUD-BASED TRAFFIC CONTROL SYSTEM FOR SUB-SAHARAN AFRICAN CITIES COMBINING CHAIN-OF-THOUGHT, VISION-LANGUAGE-ACTION MODELS AND LATENT WORLD MODELS**" under the supervision of Dr. E. P. Fasina, Department of Computer Sciences, Faculty of Computing and Informatics, University of Lagos, in partial fulfilment of the requirements for the award of the Master of Information Technology degree in Computer Science.

Dr. E. P. Fasina Supervisor	Date
Prof. F. A. Oladeji Head of Department	Date

DEDICATION

This work is dedicated to [name of family member or loved ones], and to everyone who spends hours each day in traffic on the roads of Lagos and other African cities, in the hope that better tools will give some of that time back.

ACKNOWLEDGEMENT

I give thanks to [God / personal acknowledgement] for the strength and health to complete this work.

I am grateful to my supervisor, Dr. Ebun P. Fasina, for guidance throughout this project, for the class session of 27 July 2026 in which the eight-layer deployment framework used in this dissertation was set out, and for patient and constructive comments on every draft. I also thank the members of the Machine Learning Research Group, Department of Artificial Intelligence and Robotics, University of Lagos, for discussions that shaped the design and for feedback on the earlier journal paper from which this work grew.

I thank the academic and administrative staff of the Department of Computer Sciences, Faculty of Computing and Informatics, University of Lagos, and the School of Postgraduate Studies, for their support during the programme. I am grateful to my classmates [names] for their company and encouragement, and to [name of employer or colleagues, if applicable] for their understanding while I completed this degree.

Finally, I thank my family and friends, especially [names], for their patience, prayers and support.

TABLE OF CONTENTS

Chapter 1: Introduction 1
1.1 Background 1
1.1.1 Urban traffic and informal transit in Sub-Saharan Africa 1
1.1.2 Connectivity, power and compute limits 1
1.1.3 The Lagos context and the opportunity 2
1.2 Problem statement 2
1.3 Aim 3
1.4 Objectives 3
1.5 Research questions 4
1.6 Scope and delimitations 4
1.7 Significance 5
1.8 Structure of the dissertation 6
Chapter 2: Literature Review 7
2.1 Traffic Signal Control 7
2.1.1 Fixed-time and actuated control 7
2.1.2 Max-pressure and queue-based adaptive control 7
2.1.3 Reinforcement-learning methods 8
2.1.4 Large language models for signal control 8
2.2 World Models and Model-Predictive Planning 9
2.2.1 Latent world models 9
2.2.2 Planning with the cross-entropy method 10
2.2.3 Hierarchical planning with world models 10
2.3 Vision-Language-Action Models and Chain-of-Thought for Control 11
2.3.1 Vision-language-action models 11
2.3.2 Chain-of-thought reasoning for control 12
2.3.3 Hierarchical and dual-system designs 12
2.3.4 Open vision-language models for the slow loop 13
2.3.5 Safety of embodied language-model agents 13
2.4 Explainability and the Regulatory Setting 13
2.5 Infrastructure Constraints in Sub-Saharan Africa 14
2.5.1 Connectivity, power and compute 14
2.5.2 Camera-based traffic systems in African cities 15
2.5.3 Mixed traffic and informal transit 15
2.6 Comparison of Approaches 15
2.7 Synthesis and Research Gap 16

Chapter 3: Methodology and System Design....18
3.1 Introduction....18
3.3 The two-timescale architecture....19
3.3.1 Edge perception....20
3.3.2 The fast loop....20
3.3.3 The slow loop....20
3.3.4 The subgoal interface....21
3.3.6 The explainability log....22
3.3.7 The capability-aware coordinator and the degradation tiers....22
3.3.8 Tiered communication....23
3.4 The eight-layer deployment framework....23
3.4.1 Origin and content....23
3.4.2 The cloud versus on-site video tension and its reconciliation....24
3.5 Cloud deployment architecture....25
3.5.1 Hosting and placement....25
3.5.2 Latency and cost....25
3.5.3 Security and data residency....25
3.5.4 Behaviour across the degradation tiers....26
3.6 Simulation environment....26
3.6.1 The junction....26
3.6.2 Phases....26
3.6.5 Seeds and their roles....28
3.7.1 Baselines that were built....28
3.8 Metrics....29
3.9 The world model and planner....29
3.10 Reproducibility and validity....31
3.12 Chapter summary....33
Chapter 4: Implementation, Results and Discussion....35
4.1 Implementation, stage by stage....35
4.1.1 Environment and skeleton (S0)....35
4.1.2 Junction and scenarios (S1)....35
4.1.3 Perception, metrics and baselines (S2)....36
4.1.4 Safety shield and its audit (S3)....37
4.1.5 Latent world model (S4)....38
4.1.6 CEM planner (S5)....38
4.1.7 Slow loop (S6)....39
4.1.8 Integration (S7)....39

4.1.9 Experiment tooling (S8) 39
4.2 Reproduction evidence 40
4.3 World-model accuracy 40
4.4 Controller comparison 42
4.4.1 Average waiting time 42
4.4.2 Other metrics 44
4.4.3 Paired by-seed tests 46
4.5 The contribution of the world model 48
4.6 Safety shield audit 49
4.7 Slow-loop evidence 50
4.7.1 Wiring checks with the mock backend 50
4.7.2 The 3B smoke test 51
4.8 Negative results and threats to validity 53
4.9 Discussion 54
Chapter 5: Summary, Conclusion, Recommendations and Future Work 56
5.1 Summary of the Study 56
5.2 Conclusions 57
5.3 Contributions 58
5.4 Recommendations 58
5.5 Limitations 58
5.6 Future Work 59
5.7 Open Questions for the Supervisor 60
References 61
Appendix A: Reproduction Commands 68
Appendix B: Scenario and Shield Parameters 70
Appendix C: Full Results 71
Appendix D: Deviations from the Specification 72

LIST OF FIGURES

Figure 1. The two-timescale architecture: a slow loop (vision-language-action model with chain-of-thought reasoning) sets a subgoal, a fast loop (latent world model with a cross-entropy-method planner) proposes a phase, and a rule-based safety shield corrects the proposal before it reaches the signal controller.....19

Figure 2. The eight-layer deployment framework, credited to Dr. Fasina, with the author's proposed refinements (not yet confirmed by the supervisor).....24

Figure 3. Queue prediction error of the world model and of the persistence baseline on held-out seeds 215 to 219, at one, two and three steps.....41

Figure 4. Training and validation loss of the world model by epoch.....42

Figure 5. Average waiting time by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group for every controller shown.....43

Figure 6. Average travel time by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group.....44

Figure 7. Average queue length by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group.....45

Figure 8. Throughput by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group…45

Figure 9. Average pedestrian waiting time by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group.....46

Figure 10. Paired difference in average waiting time, africot_noslow minus max-pressure, per scenario, mean over five seeds with standard error as error bars; positive values mean max-pressure is lower.....48

Figure 11. Shield corrections by rule summed over all evaluation runs, on a logarithmic scale.....49

Figure 12. Connectivity state of the coordinator over the simulation clock in the outage scenario on seed 0 with the mock backend: online, holding the last subgoal, using the cached schedule, and online again.....51

Figure 13. Renderer keyframe used for the smoke test: market scenario, tuning seed 100, 226 s into the simulation; waiting pedestrians appear as dots at the crossings, and the signal state per movement is shown at the top left.....52

LIST OF TABLES

Table 1. Approaches reviewed, compared against requirements of the target setting.....16
Table 2. The subgoal schema, default values and permitted ranges.....21
Table 3. Scenario parameters (demand per origin arm in vehicles per hour, vehicle mix, pedestrian demand and events).....27
Table 4. Specified versus built: each component of the design and its status in the prototype.....32
Table 5. Scenario parameters as implemented (vehicles per hour per origin arm; pedestrians per hour in total).....36
Table 6. Average waiting time in seconds, mean and standard deviation over five evaluation seeds.....42
Table 7. Average waiting time, average travel time, average queue length, throughput and pedestrian waiting time per scenario and controller, mean and standard deviation over five evaluation seeds.....43
Table 8. Paired by-seed differences in average waiting time (first controller minus second), with two-sided p-values; exploratory, five seeds, no multiplicity correction.....46
Table 9. Shield corrections by controller and rule, summed over all evaluation runs of that controller.....49
Table 10. Deviations from the specification recorded during the build.....53
Table 11. Scenario parameters (demand in vehicles per hour, pedestrians per hour, vehicle mix and event windows).....70
Table 12. Safety shield and controller parameters.....70
Table 13. Full results by scenario and controller (mean and standard deviation over five seeds).....71
Table 14. Deviations from the build specification.....72

ABSTRACT

Traffic signal control in Sub-Saharan African cities must cope with mixed informal transit, intermittent connectivity and power, and a need for auditable decisions. This dissertation proposes AfriCoT-VLA, a cloud-based traffic control system, with cloud-hosted reasoning and edge fallback for intermittent connectivity, following a supervisor-proposed eight-layer framework. In the intended deployment, a cloud-hosted vision-language-action model with chain-of-thought reasoning turns a compressed junction keyframe and its context, every few minutes, into a compact subgoal with a written justification. An on-site latent world model and a cross-entropy method planner choose phase timings against that subgoal, behind a rule-based safety shield that nothing learned can bypass, and keep the junction running when the link drops. A T-junction with cars, minibuses, tricycles and motorcycles was built in SUMO and examined in four scenarios over five seeds, against fixed-time, camera-count-plus-timer and max-pressure baselines. On held-out data the world model lowered one-step queue prediction error by 69.4 percent compared with a persistence guess. The planner without the slow loop lowered average waiting time against fixed-time on all three scenarios, but max-pressure was lower on normal in all five seeds, and the two were comparable on market and incident. Without the world model it was no better than fixed-time. The shield recorded 0 violations in evaluation. The full system, including the slow loop, is implemented and wiring-tested with a scripted backend and one smoke test of a 3-billion-parameter model, but it has not yet been evaluated, so no claim is made about its benefit. No cloud deployment has been built or measured, so no latency, cost, availability, security or data-residency result exists. The prototype is limited to one junction, simulated perception and five seeds, and two planned baselines are not built.

Keywords: cloud-based traffic control; traffic signal control; chain-of-thought; vision-language-action models; latent world models; Sub-Saharan Africa; safety shield

Chapter 1: Introduction

1.1 Background

Signal timing is among the few levers that a city authority can adjust without building new road space. Much of the research on adaptive signal control, surveyed by Wei et al. (2019c), has been developed under assumptions of a comparatively homogeneous vehicle fleet, lane-disciplined behaviour, dependable electricity and continuous network access. Many cities in Sub-Saharan Africa (SSA) differ on each of these points, so a controller designed for the former cannot be assumed to suit the latter.

1.1.1 Urban traffic and informal transit in Sub-Saharan Africa

Road traffic in SSA cities mixes private cars with a large informal public transport sector. In Lagos, minibuses known as danfo carry a large share of passengers, tricycles known as keke fill gaps in the road network, and motorcycle taxis known as okada weave between lanes and through queues. Comparable services operate elsewhere, including matatu minibuses in Kenya and boda-boda motorcycle taxis in East Africa. These descriptions come from the author's familiarity with the setting, not a cited survey, and no vehicle-share statistic is asserted. What matters for signal control is the behavioural consequence: vehicles of very different length, width and speed share the same approach, motorcycles do not respect lane boundaries, and public transport vehicles stop at irregular points. Queue length in vehicles is therefore a poor proxy for queue demand.

Pedestrians add a second difficulty, since a controller that ignores their waiting can reduce vehicle delay at their expense. Demand is also time-dependent: market days, events and incidents change arrivals in ways a fixed plan does not anticipate.

1.1.2 Connectivity, power and compute limits

The second group of constraints concerns the infrastructure beneath an intelligent controller. In 2023, 565 million people in SSA lacked access to electricity, which was 85 per cent of the people without access worldwide (International Energy Agency et al., 2025). Roughly 53 per cent of the SSA population had access to electricity, against roughly 92 per cent worldwide (World Bank, 2026). Mobile internet penetration in the region stood at 27 per cent at the end of 2023 (GSMA, 2024a). These regional figures do not describe any one junction, and cities are better served than the averages, but a control system cannot assume continuous power or a reliable wide-area link at every site. Data-

centre capacity on the continent is also limited (Africa Data Centres Association, 2026), so where large models are hosted is itself a design question. These limits argue for a cloud-hosted reasoning tier paired with an on-site tier that keeps working when the link fails.

1.1.3 The Lagos context and the opportunity

Lagos is the setting that motivates this work. Reports describe a phased deployment of AI traffic systems on selected corridors by the Lagos Metropolitan Area Transport Authority (ITEdgeNews, 2026), the selection of a vendor (Olomu, 2025) and public discussion of whether such systems can ease congestion (Adeyemo, 2026). Nairobi shows a parallel pattern (Mumbi, 2026; Nairobi Law Monthly, 2026). These sources indicate that investment is under way, not that any particular design performs well.

Three lines of research are relevant. Latent world models learn a compact predictive model of how a system evolves and allow a controller to plan by imagining outcomes before acting (Hafner et al., 2019; Hafner et al., 2025; Hansen et al., 2024). Vision-language-action models (VLAMs) extend large pretrained vision-language models to produce actions for an embodied agent (Zitkovich et al., 2023; Kim et al., 2024), and chain-of-thought (CoT) reasoning in such models has been explored as a way of making the path from observation to action explicit (Zawalski et al., 2024; Yang et al., 2025). In signal control, reinforcement learning is well established (Wei et al., 2019a; Chu et al., 2020) and language-model agents have begun to appear (Lai et al., 2023). The authors' earlier position paper proposed that a slow loop generating compact subgoals, coupled to a fast on-device latent world model planner, treats SSA infrastructure gaps as design inputs (Ariyo & Fasina, 2026). This dissertation applies that architecture to signal control, intends it to be cloud-hosted, and adds the elements that a regulated public safety function requires.

1.2 Problem statement

Adaptive controllers designed for well-provisioned cities are not guaranteed to suit SSA junctions. Fixed-time plans cannot respond to market days or incidents, controllers that rely on continuous cloud access stop adapting when the link fails, and large reasoning models cannot be run at every junction. Learned controllers are usually opaque, which conflicts with the need for decisions that the Lagos State Traffic Management Authority (LASTMA) or the Ministry of Transportation can audit. Finally, much of the learned-

control literature is evaluated on homogeneous traffic that excludes lane-splitting motorcycles and pedestrians.

The problem addressed here is therefore the absence of a cloud-based signal control design that (a) keeps basic, safe, locally computed control running when connectivity or power is degraded, (b) uses context such as time, events and incidents through reasoning hosted in the cloud when it is reachable, (c) records every decision in a form that can be audited, and (d) is evaluated under traffic that includes informal transit and pedestrians. Whether the design improves control over simpler alternatives is empirical, and part of that question remains open here.

1.3 Aim

The aim is to design, implement and evaluate in simulation a prototype of AfriCoT-VLA, a cloud-based traffic signal control system for SSA cities that pairs cloud-hosted chain-of-thought vision-language-action reasoning with an on-site latent world model through a compact subgoal, and to report honestly what the evidence shows about each component.

1.4 Objectives

The prototype is a single T-junction in the SUMO simulator (Alvarez Lopez et al., 2018), driven through its TraCI interface.

1. To build a simulated Lagos-style T-junction with mixed vehicle classes (car, danfo, keke, okada), pedestrian crossings and four scenarios (normal, market, incident and outage), each run for 3600 s after a warm-up of 300 s, with evaluation on seeds 0 to 4.
2. To implement a noisy perception layer and metrics for average travel time, waiting time, queue length, throughput and pedestrian waiting.
3. To implement fixed-time, camera-count-plus-timer and max-pressure (Varaiya, 2013) baselines and run them on every scenario and seed.
4. To implement a rule-based safety shield between the planner and the signals, and test that it corrects unsafe proposals.
5. To train a latent world model of queue evolution and test whether it beats a persistence guess by at least 10 per cent in one-step queue error on held-out data.
6. To implement a Cross-Entropy Method (CEM) planner (de Boer et al., 2005) that uses the world model under a subgoal-defined cost, with ablations without the slow loop and without the world model.
7. To implement the slow loop, in which a VLAM with CoT reads a rendered snapshot and context and returns a validated subgoal, with a rule-based coordinator that schedules calls, detects novelty and handles connectivity loss by holding the last subgoal and then a cached schedule. The cloud link is represented in simulation by a scripted backend and outage windows.

8. To implement an explainability log of every decision, and test the full controller with a mock backend and once with a small open-weight VLAM (Bai et al., 2025).
9. To describe, without building them, the cloud-based eight-layer framework, the cloud deployment architecture, tiered communication and a path to field deployment.

1.5 Research questions

10. **RQ1.** Can a compact latent world model of junction queues, trained on logged transitions from simple controllers, predict the next decision window's queues better than the assumption that queues remain unchanged?
11. **RQ2.** How does a CEM planner that uses this world model compare with the fixed-time, count-plus-timer and max-pressure baselines across the normal, market and incident scenarios?
12. **RQ3.** How much of the planner's behaviour depends on the world model, and how much on the subgoal, as indicated by the two ablations?
13. **RQ4.** Does the safety shield hold, in the sense that no conflicting movements are ever green together and no rule violation is recorded across all evaluated runs, and which rules intervene most often?
14. **RQ5.** Does the slow-loop and coordinator machinery behave as specified, in that calls are scheduled, novelty triggers fire, outages lead to hold-last and then cached-schedule behaviour, and every decision is logged?
15. **RQ6.** Does the slow loop with CoT reasoning change control relative to the planner without it, and is a small open-weight VLAM's reasoning reliable enough to use?

RQ1 to RQ5 can be answered from the evidence here. RQ6 can be answered only in part: a single smoke test of a small model gives some evidence about the reliability of its reasoning, but the full controller, africot_full, is implemented and wiring-tested and has not been evaluated on the scenario grid. A faithful evaluation calls for a larger open-weight model on a GPU, which was not available.

The hypothesis behind the slow loop is that a model which can read context unavailable to the planner, such as an incident on one approach or a market day, could shift the planner's cost weights in a useful direction. This is a design intention and not a finding. The evaluation of africot_full on the scenario grid is pending, and no statement about its effect on control is made in this dissertation until it is complete.

1.6 Scope and delimitations

First, the system is intended to be cloud-hosted, but no cloud deployment has been built or tested. The prototype simulates the control logic and outage behaviour only, with a scripted backend and one CPU smoke test of a 3B model on a local machine. No latency, cost, availability, security or data-residency measurement exists.

Second, the evaluation covers one isolated T-junction; the multi-junction corridor, neighbour information, green waves and mesh relay remain proposals.

Third, the evidence is from simulation only. SUMO traffic and demand approximate Lagos traffic and were not calibrated against field counts, and perception is simulated by adding noise to ground truth, so no real detector is evaluated. No field trial was carried out.

Fourth, three of the five planned baselines were built. A deep reinforcement learning controller and LLMLight (Lai et al., 2023) were not, which limits any comparison with learned alternatives.

Fifth, africot_full is implemented and wiring-tested with a mock backend, and a small VLAM has been run once on a CPU. It is not evaluated, and mock runs are never reported as VLAM results.

Sixth, the market side-road demand was lowered from 550 to 200 vehicles per hour after the specified value produced gridlock under fixed-time control. The outage scenario differs from market only in the slow-loop connectivity window, so controllers without a slow loop yield identical rows for the two.

Seventh, the emergency override, the tiered communication channels, cryptographic message signing and the fixed-time fallback for a system fault are design only. With five seeds per cell, paired by-seed tests are exploratory, and no significance claim is made where a gap lies within one standard deviation.

Finally, the models are open-weight and none is trained from scratch. The world model is a small network in the style of DreamerV3 (Hafner et al., 2025) and TD-MPC2 (Hansen et al., 2024), not either of them.

1.7 Significance

The study is intended to be useful in four ways. For traffic authorities and researchers in the region, it offers an open, reproducible simulation testbed with informal transit, pedestrians, market and incident scenarios and an outage condition, with every controller decision logged for audit. For embodied AI design, it offers a worked example of coupling a cloud-hosted reasoning model to a fast on-site planner through a small subgoal vector, behind a shield that nothing learned can bypass. For the research community, it reports negative and null results as found. For the authors, it provides a base for the

corridor extension, the missing baselines, the GPU evaluation and the measurement of a real cloud deployment.

The eight layers were set by the supervisor, Dr. Ebun P. Fasina, in the class session of 27 July 2026, when they typed into DeepSeek on a shared screen the prompt "Design a cloud-based 8-layer framework for AfriCoT-VLA". The framework is cloud-based, with compressed video sent to the cloud. The system proposed here is intended to follow it: the cloud is the primary tier for the data store, model, explainability, service, agent and autonomous layers, and the network layer carries compressed keyframes for the slow loop rather than continuous raw video, which never leaves the site. The fast loop and the safety shield stay on site as the resilience tier, so the junction keeps operating when the link drops. The methodology chapter develops this reconciliation. Its details and the refinements to individual layers are the author's proposals, not yet confirmed by the supervisor, and are not evaluated.

1.8 Structure of the dissertation

The dissertation has five chapters. The second chapter reviews the literature on traffic signal control, from fixed-time plans through max-pressure, reinforcement learning and language-model agents, then on world models, vision-language-action models and chain-of-thought reasoning, explainability and regulation, and regional infrastructure constraints, and identifies the gap addressed.

The third chapter presents the methodology and system design: the two-timescale architecture, the subgoal interface, the safety shield, the explainability log, the coordinator and degradation tiers, the tiered communication scheme, the eight-layer framework and the cloud deployment architecture. It also describes the simulation setup, scenarios, baselines, metrics and reproducibility measures, and contains a table separating what was specified from what was built.

The fourth chapter reports the implementation, the world model against persistence, the controller comparison, the shield audit, the wiring evidence for the slow loop, the smoke test, the negative results and the threats to validity. The fifth chapter summarises the findings, draws conclusions, makes recommendations and sets out future work, including cloud deployment and measurement, the corridor extension, the missing baselines and the staged deployment roadmap. References and appendices follow.

Chapter 2: Literature Review

This chapter reviews the work on which the proposed traffic-control system builds: traffic signal control, from fixed-time plans to control by large language models; latent world models and planning; vision-language-action models and chain-of-thought reasoning; explainability and the regulatory setting; and the infrastructure constraints of Sub-Saharan Africa (SSA). It closes with a synthesis that states the gap the dissertation addresses. Where the literature does not support a claim, the claim is omitted or is stated as the author's own reasoning.

2.1 Traffic Signal Control

2.1.1 Fixed-time and actuated control

The simplest form of signal control repeats a pre-computed cycle whose phase durations are set from historical demand. Fixed-time plans need no sensing and little computation, which helps to explain their persistence in cities with limited detector coverage. Their weakness is that the plan cannot respond to demand that departs from its assumptions, such as a market day, a stalled vehicle or a burst of motorcycle traffic. Actuated control relaxes this by extending or terminating a phase according to detector calls, within bounds set by the engineer. Surveys treat these two families as the conventional baseline against which adaptive and learning-based methods are compared (Wei et al., 2019c). Both continue to operate when communication fails, because nothing outside the junction is required. The proposed system retains a fixed-time plan for exactly this reason, as the last tier of its degradation ladder.

2.1.2 Max-pressure and queue-based adaptive control

Max-pressure control is a decentralised policy for a network of signalised intersections that chooses, at each decision point, the phase that relieves the greatest pressure, where pressure is the difference between the queue upstream of a movement and the queues on the downstream links it feeds (Varaiya, 2013). The method needs only local queue information, requires no knowledge of demand, and is accompanied by an analytical stability result under the model assumptions of the original work. Because it is inexpensive and well understood, it is a standard non-learning adaptive baseline, and it is the reference against which the controllers in this dissertation are compared. Its limits are visible from its definition: it optimises a queue-difference quantity rather than travel time

or the priority of particular road users, and it cannot take into account context such as a market, an event or an incident.

Wei et al. (2019a) used the pressure concept to design the state and reward of a reinforcement-learning agent, which links the analytic and learned traditions. The slow-to-fast interface in the methodology chapter likewise uses weights on queue-related terms, not phase commands.

2.1.3 Reinforcement-learning methods

Reinforcement learning (RL) treats signal control as a sequential decision problem in which an agent observes a traffic state, selects a phase or duration, and receives a reward derived from queues, delay or throughput. Wei et al. (2019c) survey this literature by state representation, action definition and reward design. CoLight introduced graph attention so that an intersection can weigh the influence of its neighbours (Wei et al., 2019b), and Chu et al. (2020) developed a multi-agent scheme intended for large-scale networks. PressLight couples learning with the pressure concept to coordinate signals along an arterial (Wei et al., 2019a).

Three properties matter here. First, learned policies are opaque: the reason for a chosen phase cannot be read from the weights in a form suitable for a regulator. Second, they are trained for a particular demand distribution, so behaviour under an unfamiliar scenario is not guaranteed. Third, the cited works treat connectivity, power and detector availability as given rather than as design inputs. These observations are the author's reading of the literature, not findings reported by it. The dissertation does not build a deep RL controller; its absence is disclosed in the methodology and results chapters and carried into the future work.

2.1.4 Large language models for signal control

The most recent line of work uses large language models (LLMs) as the controller. LLMLight presents the LLM as a signal control agent that receives a textual description of the traffic state and produces a phase decision together with its reasoning (Lai et al., 2023). A decision accompanied by language can be inspected, but this design has a difficulty. A model that issues the phase command itself must respond at the decision frequency of the junction, and a large model reached over a network cannot be assumed to be available at every decision. Latency, cost and dependence on a remote service are

properties of this placement, and the cited work does not frame them as constraints of low-connectivity deployments.

This dissertation therefore adopts LLM-style reasoning about traffic context but not its placement in the control loop. The reasoning model runs infrequently and emits a compact subgoal with a justification, while a lightweight planner on site chooses phase timing. LLMLight is a natural comparison for future work and, as stated in the introduction, is not among the controllers evaluated here.

2.2 World Models and Model-Predictive Planning

2.2.1 Latent world models

A world model is a learned model of how the environment changes in response to actions. When it operates in a compact latent space rather than on pixels, it can be rolled forward cheaply, allowing an agent to evaluate candidate action sequences without acting. PlaNet learned a latent dynamics model and planned within it from pixels (Hafner et al., 2019). Chua et al. (2018) showed that probabilistic dynamics models combined with model-predictive control can perform well with relatively few interactions, which is attractive wherever real data are scarce. MuZero extended the principle to models that predict only the quantities needed for planning (Schrittwieser et al., 2020).

DreamerV3 learns a world model and trains behaviour from imagined latent trajectories, and its authors report that a single hyperparameter configuration was applied across a wide range of control domains (Hafner et al., 2025). TD-MPC2 combines a latent, decoder-free world model with planning at inference time (Hansen et al., 2024). V-JEPA 2 is a joint-embedding predictive model trained by self-supervision on video and adapted with a comparatively small amount of robot interaction data (Assran et al., 2025). DayDreamer reported that Dreamer-style models can be trained directly on physical robots (Wu et al., 2022), and Dynalang learns to predict future observations and text jointly (Lin et al., 2024).

For an edge deployment, size and cost matter, and the cited works do not, in general, report the budget of a roadside computer. The dissertation takes the position, as a design choice and not a literature finding, that a small latent model trained on junction data is the appropriate scale for the fast loop, and that the larger models named above are better treated as references than as deployment components. Under the open-weight constraint,

DreamerV3 and TD-MPC2 are the candidate families for the world model, while V-JEPA 2 is regarded as a heavier option suited to offline training.

2.2.2 Planning with the cross-entropy method

Model-predictive control selects an action by optimising predicted outcomes over a short horizon, executing the first part and then replanning. In learned-model settings the optimiser is commonly a sampling method. The cross-entropy method (CEM) draws candidate action sequences from a parametric distribution, evaluates them, keeps the best fraction, and refits the distribution to that elite set, repeating for a small number of iterations (Rubinstein, 1999; de Boer et al., 2005). It needs no gradients through the model, handles non-differentiable objectives, and its cost is controlled directly by the number of samples and iterations. PlaNet used CEM within a latent model (Hafner et al., 2019), Chua et al. (2018) used sampling-based trajectory optimisation in the same spirit, and TD-MPC2 likewise plans at inference time (Hansen et al., 2024).

For signal control, three features of CEM are useful. The objective can be a weighted sum of queue, delay and pedestrian terms, so the weights can be changed without retraining the model. The computation budget is predictable, which matters on constrained hardware. The candidate plans are explicit phase timings, which can be passed through a rule-based check before they are applied. These features are the reason the proposed system exposes cost weights and constraint bounds as the subgoal interface to the planner.

2.2.3 Hierarchical planning with world models

Director is a hierarchical agent in which a manager operating in the latent space of a world model selects goals and a worker is trained to achieve them (Hafner et al., 2022). Its relevance is structural: a slow component can choose goals for a fast component without issuing the low-level actions itself, which is the arrangement adopted here. Here the slow component is a language-capable model that reads context outside the sensor stream.

Other work joins generative models to action generation more tightly. UniPi casts decision-making as text-conditioned video generation followed by an inverse-dynamics step (Du et al., 2023), SuSIE uses an image-editing diffusion model to propose subgoal images (Black et al., 2024), RoboDreamer learns compositional world models for imagination (Zhou et al., 2024), and 3D-VLA couples a three-dimensional VLA model with a generative world model (Zhen et al., 2024). These works address robotics, and

their results are not assumed to carry over to traffic. A recurring theme in position and survey papers is that perception, reasoning and prediction are developed separately and that a unified model is a long-term aim (Feng et al., 2025; Xu, 2025; Gupta et al., 2024). The present work treats full unification as an open research problem, a view that the authors' earlier manuscript also adopts (Ariyo & Fasina, 2026), and deliberately couples two separate models through a narrow interface.

2.3 Vision-Language-Action Models and Chain-of-Thought for Control

2.3.1 Vision-language-action models

Vision-language-action (VLA) models extend a pretrained vision-language model so that it outputs actions. RT-2 showed that web-scale vision-language knowledge could transfer to robotic control by expressing actions as tokens (Zitkovich et al., 2023); its weights are not open, so it is cited as related work and is not a candidate component. The Open X-Embodiment collection showed the benefit of pooling data from many robot embodiments (Open X-Embodiment Collaboration, 2024). Octo is an open generalist policy (Octo Model Team et al., 2024), and OpenVLA is an open-weight VLA that its authors report as outperforming a much larger closed model on their evaluation tasks, while supporting parameter-efficient fine-tuning (Kim et al., 2024). OpenVLA-OFT was proposed to improve fine-tuning speed and task success (Kim et al., 2025), π0 pairs a vision-language backbone with a flow-matching action model (Black et al., 2025), and GR00T N1 is an open foundation model for humanoid robots (NVIDIA et al., 2025). Surveys document the breadth of this literature (Sapkota et al., 2025; Li et al., 2025), the wider background of foundation models is set out by Bommasani et al. (2021), and CLIP, Flamingo, PaLM-E, Gato and VIMA supply the lineage on which these models rest (Radford et al., 2021; Alayrac et al., 2022; Driess et al., 2023; Reed et al., 2022; Jiang et al., 2023).

Two observations follow. The VLA literature is almost entirely concerned with manipulation and humanoid control, so its models are trained on arm and hand action spaces rather than signal phases, and adopting a VLA for traffic cannot mean using a manipulation checkpoint as a controller. Most of these models are also large and are reported on server-class processors rather than roadside hardware. For SSA deployment, this is a reason to place the VLA-style model in a slow loop that runs rarely, and not in the fast loop.

2.3.2 Chain-of-thought reasoning for control

Embodied chain-of-thought (ECoT) trains a VLA to produce intermediate reasoning steps before it outputs an action, including a plan, subtasks, motion descriptions and object locations, and its authors report improved generalisation of OpenVLA-style policies on the tasks they studied (Zawalski et al., 2024). InstructVLA addresses a related tension: instruction tuning for manipulation can erode the multimodal reasoning of the underlying model, and its training recipe is intended to keep understanding and action together (Yang et al., 2025). Both show that a written trace can accompany control, which supports explainability, at a cost in time.

In ECoT the reasoning precedes every low-level action, so its latency is paid at the control rate. In the proposed system the reasoning is generated on the slow timescale, where seconds or tens of seconds are acceptable, and its output is a justification plus a subgoal, stored for audit. These works do not guarantee the reliability of traces produced by small models, and the dissertation reports its own smoke test of a small open model in the results chapter rather than assuming reliability.

2.3.3 Hierarchical and dual-system designs

Several systems split the problem into a slow, deliberative component and a fast, reactive one. Hi Robot places a high-level vision-language model above a low-level VLA, with the upper model producing language instructions and responding to user feedback (Shi et al., 2025). Helix, a technical report from a commercial developer, describes a slow system for scene understanding and a fast system for continuous control within one humanoid platform (Figure AI, 2025). GR00T N1 is likewise described with two cooperating modules (NVIDIA et al., 2025), and Director supplies the counterpart on the world-model side (Hafner et al., 2022).

These designs establish that the slow and fast split is a recognised pattern, and the dissertation does not claim novelty for the split itself. What the cited works do not provide, as far as the verified literature reviewed here shows, is a version designed around SSA infrastructure constraints, with an explicit fallback when the slow component is unreachable, a rule-based safety layer between planner and actuator, and an audit trail of reasoning intended for a transport regulator. The authors' earlier manuscript set out the low-resource rationale for the two-timescale pattern as a design proposal (Ariyo & Fasina, 2026).

2.3.4 Open vision-language models for the slow loop

The slow loop requires a model that reads images and text. Qwen2.5-VL is an open-weight vision-language model family released in several sizes (Bai et al., 2025). A small variant makes an on-site or cluster-level slow loop conceivable on modest hardware, while a larger variant could be hosted in the cloud when connectivity allows, which is the basis of the degradation tiers described in the methodology chapter. The cited technical report evaluates general benchmarks and does not support the statement that a given variant is adequate for traffic reasoning. Whether a small model reasons reliably about a junction is an empirical question that the dissertation examines only through a limited smoke test.

A cloud-hosted slow loop with an edge fallback is the placement this dissertation intends. The reasoning model, the data store and the audit log are intended to be hosted on the cloud, in line with the supervisor's cloud-based framework, while the fast loop and the safety shield remain on site so that the junction keeps operating when the link drops. The literature reviewed here does not evaluate such a split for signal control, and the placement is therefore presented as a design proposal.

2.3.5 Safety of embodied language-model agents

If a language-capable model is placed anywhere near actuation, its failure modes matter. SafeAgentBench is a benchmark for safe task planning by embodied LLM agents, introduced because such agents can produce unsafe plans (Yin et al., 2024), and SafeEmbodAI proposes a safety framework for mobile robots in embodied AI systems (Zhang et al., 2024). Both are preprints concerning settings other than traffic, but they are consistent with a principle adopted here: learned components should not be the last line of defence. A rule-based shield enforces minimum green, amber, all-red clearance and pedestrian crossing time between the planner and the signal controller, and nothing learned can bypass it.

2.4 Explainability and the Regulatory Setting

Explainability in traffic control serves operators and a public authority that reviews whether behaviour was consistent with its rules. The project context names the Lagos State Traffic Management Authority (LASTMA) and the Ministry of Transportation as the audit stakeholders, and the design therefore keeps a timestamped record of each decision together with the reasoning trace and the triggering conditions. This chapter does

not report the content of any regulation or audit procedure of either body, and the design should be reviewed with them before any claim of regulatory suitability is made.

The literature offers three relevant observations. Learned policies that output only an action, as is typical in RL signal control, give no record of their reasoning (Wei et al., 2019c). Max-pressure is transparent in the sense that its rule can be stated in one sentence (Varaiya, 2013), but it cannot explain a context-dependent departure from that rule. Language-based controllers and chain-of-thought VLAs produce readable traces (Lai et al., 2023; Zawalski et al., 2024), but a readable trace is not necessarily a faithful account of the computation. The dissertation therefore treats a trace as a record for audit and human review, not as proof of correct reasoning, and pairs it with the shield so that safety does not depend on the trace being right.

Counterfactual analysis is a further route to explanation that a world model makes available in principle: because the model can be rolled forward under alternative actions, it can be asked what would have happened had a different phase plan been chosen. The cited world-model literature uses imagination for planning and not for explanation (Hafner et al., 2025; Hansen et al., 2024), so the use of rollouts for what-if explanation is a proposal of this work. The framework originates with Dr. Fasina, who proposed the eight layers in the class session of 27 July 2026, and the author's refinements are described in the methodology chapter.

2.5 Infrastructure Constraints in Sub-Saharan Africa

2.5.1 Connectivity, power and compute

Most of the work reviewed above assumes dependable connectivity, ample compute and stable power. The available regional statistics indicate that these assumptions are weak across much of SSA. The GSMA reported mobile internet penetration in Sub-Saharan Africa of 27% by the end of 2023 (GSMA, 2024a), and its global report (GSMA, 2024b) provides the worldwide comparison. On electricity, the World Bank indicator records that about 53.3% of the SSA population had access in 2023, against about 91.6% worldwide (World Bank, 2026), and the multi-agency energy progress report states that 565 million people in the region lacked electricity in that year, about 85% of the global total (International Energy Agency et al., 2025; United Nations Statistics Division, 2025). The Africa Data Centres Association places the continent's share of global data-centre capacity at 0.6% (Africa Data Centres Association, 2026). A recent analysis of embodied

AI for science in the Global South argues that methods must be shaped around the available compute, connectivity and energy (Liu et al., 2026).

These figures describe populations and regions, not individual junctions. A junction in a major city may have better power and coverage than the regional average, so the statistics show only that the assumptions are unsafe, not the state of any site. This is why the dissertation treats connectivity, power and compute as design inputs and builds the control loop so that its safety-critical part runs on site.

2.5.2 Camera-based traffic systems in African cities

Cities in the region are investing in camera-based and centrally coordinated signal systems. Public reports describe the selection of a vendor for an intelligent traffic system in Lagos (Olomu, 2025), a phased deployment of AI-based traffic systems by the Lagos Metropolitan Area Transport Authority (ITEdgeNews, 2026), and commentary on whether AI could help with congestion in Lagos (Adeyemo, 2026). In Kenya, the Kenya Urban Roads Authority (2023) has published on intelligent transport systems, and the press has reported on Nairobi programmes and tenders for AI-enabled signals (Kinyanjui, 2024; Mumbi, 2026; Nairobi Law Monthly, 2026). These sources are news and agency reports, cited only as evidence that interest and procurement activity exist. They do not describe the control algorithms used.

2.5.3 Mixed traffic and informal transit

The project context identifies a traffic mix that includes okadas, kekes, danfo, matatus and boda-boda, with motorcycle lane-splitting among the behaviours of interest. The verified reference set does not include a source that characterises this mix quantitatively for the target corridor, and this chapter makes no quantitative claim about it. The studies cited above appear to be framed around conventional vehicles, so mixed traffic in simulation is treated as a disclosed limitation.

2.6 Comparison of Approaches

The table below summarises how the families of approach reviewed here relate to the requirements of the target setting. The entries record what the cited works describe or what follows from their design; where the works are silent on a requirement, the table says so, not that the approach fails it.

Table 1. Approaches reviewed, compared against requirements of the target setting

Approach	Representative sources	What it uses to decide	Readable reasoning record	Behaviour when a remote service is unreachable	Treats SSA constraints as a design input
Fixed-time	Wei et al. (2019c)	Pre-set plan from historical demand	The plan itself	Continues unchanged	Not a stated focus
Actuated	Wei et al. (2019c)	Detector calls within set bounds	Rule can be stated	Continues locally	Not a stated focus
Max-pressure	Varaiya (2013)	Upstream and downstream queue difference	Rule can be stated	Local to junction and neighbours	Not a stated focus
Deep RL for signals	Wei et al. (2019a, 2019b); Chu et al. (2020)	Learned policy over traffic state	Not provided by the policy	Depends on where the policy is deployed	Not a stated focus
LLM signal agent	Lai et al. (2023)	Language description of traffic state	Textual reasoning produced	Not addressed in the cited work	Not a stated focus
Latent world model with CEM planning	Hafner et al. (2019, 2025); Hansen et al. (2024); Rubinstein (1999)	Imagined rollouts scored by an objective	Objective and plan are explicit; no language	Planner can run locally	Not a stated focus
VLA with chain-of-thought	Zawalski et al. (2024); Kim et al. (2024)	Image and language to reasoning and action	Textual reasoning produced	Not addressed; models are large	Not a stated focus
Hierarchical or dual-system	Hafner et al. (2022); Shi et al. (2025); Figure AI (2025)	Slow goal-setting above fast control	Language at the upper level in some designs	Not addressed in the cited works	Not a stated focus
Proposed design (this dissertation)	Methodology and implementation chapters	Slow context-reading loop sets weights for a local planner behind a shield	Timestamped reasoning trace per decision	Fast loop continues; tiered fallback to fixed-time plan	Yes, by design

The last row describes intent, not demonstrated performance. Its slow-loop component is implemented and wiring-tested but not yet evaluated, as the results chapter reports.

2.7 Synthesis and Research Gap

Four threads emerge from the review. Signal control has moved from fixed rules to learned and language-based controllers, but the more capable the controller, the less likely it is to be both explainable and independent of remote services. Latent world models with sampling-based planning provide a local, inexpensive, objective-driven way to choose actions, with weights that can be changed without retraining. VLA and chain-of-thought work shows that reasoning traces can accompany control, at a latency cost that suits a slower timescale. Safety work on embodied language agents is consistent with a rule-based layer that nothing learned can override.

Against this background the following gap is identified. The literature treats these threads separately. Signal-control studies address junctions but do not take language-based context or infrastructure scarcity as design inputs. VLA and world-model studies address reasoning and prediction but concern manipulation and humanoid robots, and assume resources that are not safe to assume in SSA. No source in the verified set combines a slow, language-capable context reader, a fast local world-model planner, a rule-based safety shield, a timestamped reasoning log for audit and an explicit degradation path for loss of connectivity within a single junction or corridor design. This statement is limited to the sources reviewed here, and it is not a claim that no such work exists.

The dissertation responds with a proposal and a limited test. The proposal is a coupled two-timescale design, with the VLAM and the latent world model kept as separate models linked only by a compact subgoal vector, since full unification remains an open research problem and coupling is a deliberate choice under the compute and connectivity limits described above. The test is a simulation study at a single junction using SUMO (Alvarez Lopez et al., 2018), comparing the fast-loop components against fixed-time, count-plus-timer and max-pressure baselines, with the slow loop implemented but not yet evaluated. The missing baselines, deep RL and LLMLight, are identified as future work.

Chapter 3: Methodology and System Design

3.1 Introduction

This chapter sets out how the AfriCoT-VLA system is designed and how the prototype was built and evaluated, keeping separate the full design, including parts specified but not built, and what was implemented and tested. The system is intended to be hosted on the cloud, with an on-site fast loop and safety shield as a resilience tier. The prototype is a single Lagos-style T-junction in the SUMO traffic simulator (Alvarez Lopez et al., 2018), whereas the design addresses a corridor of nearby junctions, tiered communication and a cloud deployment. A table near the end lists the status of every component.

The design treats the constraints of Sub-Saharan African cities as inputs to the architecture. Five principles follow.

First, the real-time control loop must never depend on a network link. Mobile data coverage in the region remains uneven (GSMA, 2024a, 2024b) and access to electricity incomplete (International Energy Agency et al., 2025). The reasoning and management layers are therefore intended to be hosted on the cloud, where a large model can run, while the loop that selects timings stays on the junction's edge computer as a resilience tier.

Second, the two kinds of reasoning involved differ in time scale and cost. Predicting queues and choosing a phase timing is fast and repetitive, whereas noticing that a market is in progress or that a lane is blocked is slow and occasional, and suits a large vision-language model. Hierarchical designs with a slow language-conditioned level and a fast control level have precedent in robotics (Shi et al., 2025; Hafner et al., 2022), and the authors' earlier paper proposed a related architecture (Ariyo & Fasina, 2026), which this chapter extends.

Third, nothing learned may bypass a safety rule: minimum green, clearance and pedestrian crossing time are explicit rules between the planner and the signals, and the slow loop can alter priorities but cannot issue phase commands. Fourth, the system degrades gracefully through four operating tiers, so that loss of connectivity or of a component leads to simpler and not unsafe behaviour.

Fifth, decisions must be explainable to regulators such as the Lagos State Traffic Management Authority (LASTMA) and the state Ministry of Transportation, so every decision is designed to be logged with its triggering conditions and, for slow-loop

decisions, the written reasoning behind the subgoal. Chain-of-thought reasoning has been used to expose intermediate steps in robot control (Zawalski et al., 2024; Yang et al., 2025).

Two further choices apply. Only open-weight models are used, none trained from scratch. And the vision-language-action model and the latent world model are coupled and not fused, linked only by a compact subgoal, because full unification of action generation and world modelling is an open research problem.

3.3 The two-timescale architecture

The architecture has an edge perception stage, a fast loop, a slow loop, a subgoal interface between them, a safety shield, an explainability log and a coordinator. Edge perception, the fast loop and the shield form the on-site resilience tier, and the slow loop with the upper management layers is intended to be hosted on the cloud. In the figure below, solid arrows carry control information and dashed arrows carry keyframes, coordination decisions and log records.

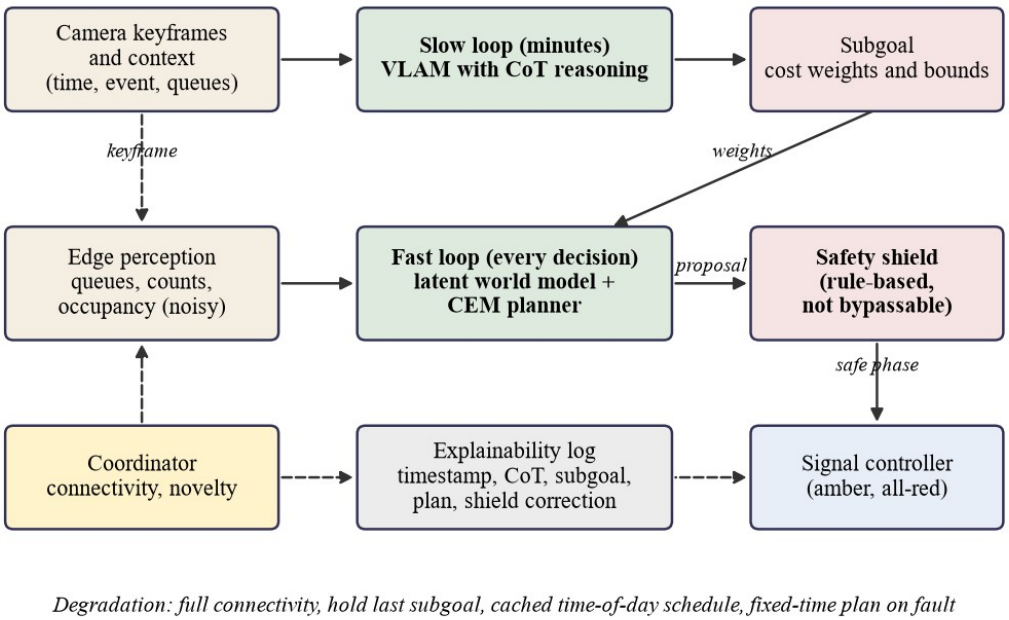


Figure 1. The two-timescale architecture: a slow loop (vision-language-action model with chain-of-thought reasoning) sets a subgoal, a fast loop (latent world model with a cross-entropy-method planner) proposes a phase, and a rule-based safety shield corrects the proposal before it reaches the signal controller

3.3.1 Edge perception

In the design, a lightweight detector on each junction's edge computer classifies vehicles by type (car, danfo, keke, okada), detects pedestrians, and computes for each approach the queue length, counts by class and occupancy, plus the pedestrians waiting at each crossing. Raw video never leaves the site; the fast loop uses only these summaries and the slow loop only occasional keyframes.

In the prototype there is no camera and no detector. Perception is simulated from SUMO ground truth with the noise model described later, and the detector is proposed for separate testing on real Lagos footage.

3.3.2 The fast loop

The fast loop runs on the junction's edge computer at every signal decision, which in the prototype is the end of each green, about 199 decisions per run on average. A latent world model predicts how queues, waiting pedestrians and departures evolve under a candidate phase and green time. The planner samples candidate three-step plans, imagines each with the world model, scores them with a cost defined by the current subgoal, and refines its sampling towards the lowest-cost plans (Rubinstein, 1999; de Boer et al., 2005; Hafner et al., 2019; Chua et al., 2018; Hansen et al., 2024). The fast loop runs locally in every operating tier and is never routed to the cloud.

3.3.3 The slow loop

The slow loop is designed to run every few minutes, or earlier when something unusual occurs, and in the intended deployment it runs in the cloud. A vision-language-action model (VLAM) receives a keyframe of the junction and a short context (time of day, any known event such as a market day or incident, current queues and the subgoal in force), reasons step by step and emits a subgoal in a fixed format. The prototype has two backends: a deterministic, rule-based `mock` backend used only for tests and wiring checks, and an `hf` backend that loads a Qwen2.5-VL model (Bai et al., 2025) through Hugging Face transformers, with the 3B model for CPU tests and the 7B model intended for a GPU run. Reasoning and answer are placed in `<reasoning>` and `<subgoal>` tags. The subgoal format is the only contract, so the design does not depend on this model, although manipulation-oriented models such as OpenVLA (Kim et al., 2024) would need adaptation.

A renderer produces the keyframe in the prototype. It draws a top-down 512 by 512 image from simulator state, with vehicles as shapes coloured by class, waiting pedestrians as dots, the signal state and a legend. It is a simulator view and not a photograph, so the prototype cannot show how a model reads real footage.

3.3.4 The subgoal interface

The subgoal is the only interface between the two loops: a compact vector of cost weights and constraint bounds that parameterise the planner's cost, so the VLAM's "action" is a statement of priorities and not a phase command. The schema is in the table below.

Table 2**.** The subgoal schema, default values and permitted ranges

Field	Meaning	Default	Range
`queue_weights` (east, west, south)	Priority of clearing each approach	1.0 each	0.2 to 3.0 each
`pedestrian_weight`	Priority of reducing pedestrian waiting	1.0	0.0 to 3.0
`max_wait_s`	Longest acceptable wait (seconds)	120	45 to 180
`transit_priority`	Priority of waiting danfos	1.0	0.0 to 2.0
`valid_for_s`	Intended validity (seconds)	300	60 to 900

A validator clamps every number to its range. A missing field keeps its previous value, and an unparseable reply leaves the previous subgoal in force.

The safety shield sits between the planner, or any other controller, and the signals. In the prototype every controller, including the baselines, passes through it. It applies these rules in order.

16. Starvation guard: an approach or crossing with observed demand and no green for over 180 s is served next.
17. Maximum green: a phase may not exceed 90 s in total.
18. Minimum green: a new phase lasts at least 10 s.
19. Pedestrian minimum: a phase that gives green to a crossing lasts at least the crossing length divided by 1.2 m/s plus 4 s, which is 16 s for the east and west crossings and 10 s for the south crossing.
20. Clearance: every phase change passes through an amber of 3 s and an all-red of 2 s, inserted by the simulation environment.

Each correction is returned with its rule and reason for logging. The shield sees only the noisy perception output and its own signal history, while a separate audit class reads simulator ground truth and counts rule violations. Because both share the same rule

constants, the audit tests the implementation and not whether the rules suffice for real-world safety.

Two departures from the design are recorded. Because perception misses pedestrians, the prototype applies the pedestrian minimum on every phase that gives green to a crossing and not only when pedestrians are observed, which is stricter and never less safe. And the spillback rule, which forbids releasing traffic towards a full downstream approach, has no meaning at a single junction and is not built.

3.3.6 The explainability log

Each decision produces one JSON line holding the simulation time, controller, scenario and seed; the trigger (scheduled, novelty or none); the connectivity state; the VLAM backend; the reasoning text; the subgoal in force; the planned phases and greens; the shield correction and its reason; the applied phase and green; and the world model's one-step error. A frame hash, a predicted-versus-actual record and tamper-evident storage are proposed and not built, and no regulator has assessed the log.

3.3.7 The capability-aware coordinator and the degradation tiers

The design includes a rule-based, non-learned coordinator that routes only slow-loop tasks according to connectivity, available compute and scene novelty. A routine scene goes to a small vision-language model on the edge, an unusual scene to a large cloud model when connectivity allows, and an emergency triggers a direct override. The four operating tiers (cloud VLAM; edge VLM or cached schedule; last subgoal; fixed-time plan) are mapped to cloud behaviour in the cloud deployment architecture below.

The prototype implements a subset. The coordinator calls the slow loop every 300 simulated seconds and early when a novelty trigger fires, namely when the mean one-step world-model queue error over the last 10 decisions exceeds twice the median of the 30 before them, with at least 120 s between triggers. During an outage window it holds the last subgoal, and after more than 600 s offline it switches to a hand-written, untuned time-of-day schedule. The slow loop is treated as instantaneous in simulated time, an idealisation since one 3B call takes minutes on a four-core CPU.

Much is not built. There is no routing between an edge model and a cloud model, the weak-connectivity tier is represented only by the cached schedule, and no automatic switch to the fixed-time plan on a fault exists. An emergency-vehicle override, in which a

verified message forces a phase directly while still respecting clearance rules, is described only.

3.3.8 Tiered communication

The design arranges five communication tiers in order of preference: 4G or fibre for everything except raw video; SMS over GSM for alerts and subgoals when data service is down; store-and-forward for logs and training data; a junction-to-junction mesh over Wi-Fi or LoRa as the main offline path, since the junctions are close together; and, optionally, satellite for critical alerts. All incoming messages are to be cryptographically signed, and may change priorities but never bypass the shield. None of these tiers is implemented.

The design also anticipates a corridor of two to three junctions, in which each world model takes its neighbours' outflows as inputs, neighbours share queue and plan data over the mesh so that green waves continue offline, and an optional cluster edge server runs the small vision-language model. The prototype has a single junction, so none of this is built.

3.4 The eight-layer deployment framework

3.4.1 Origin and content

The layered framework used here originates with the supervisor, Dr. Fasina, and is the organising structure of the intended cloud deployment. During the class session of 27 July 2026 they typed the prompt "Design a cloud-based 8-layer framework for AfriCoT-VLA" into DeepSeek on a shared screen, and the structure is credited to them. The layers are the sensor layer (cameras and actuators for traffic lights); the network connection (compressed video from the cameras over the internet to the cloud); the data store; the model layer (chain-of-thought reasoning, latent world models and the VLAM); the explainability layer; the service layer; the AI agent layer; and the autonomous agentic layer.

The author proposes refinements, which are the author's own and have not yet been confirmed by the supervisor. The data store would hold a hot window and a cold history. The model layer would follow the two-timescale design of this chapter. The service layer would expose decision, query, override and health interfaces. The AI agent layer would handle corridor coordination, and the autonomous agentic layer would produce plan proposals and what-if simulations that a human signs off. Any layer above the model

layer could fail with fallback to fixed-time control, the explainability layer would act as a deployment gate, and the network layer is proposed to be the real bottleneck.

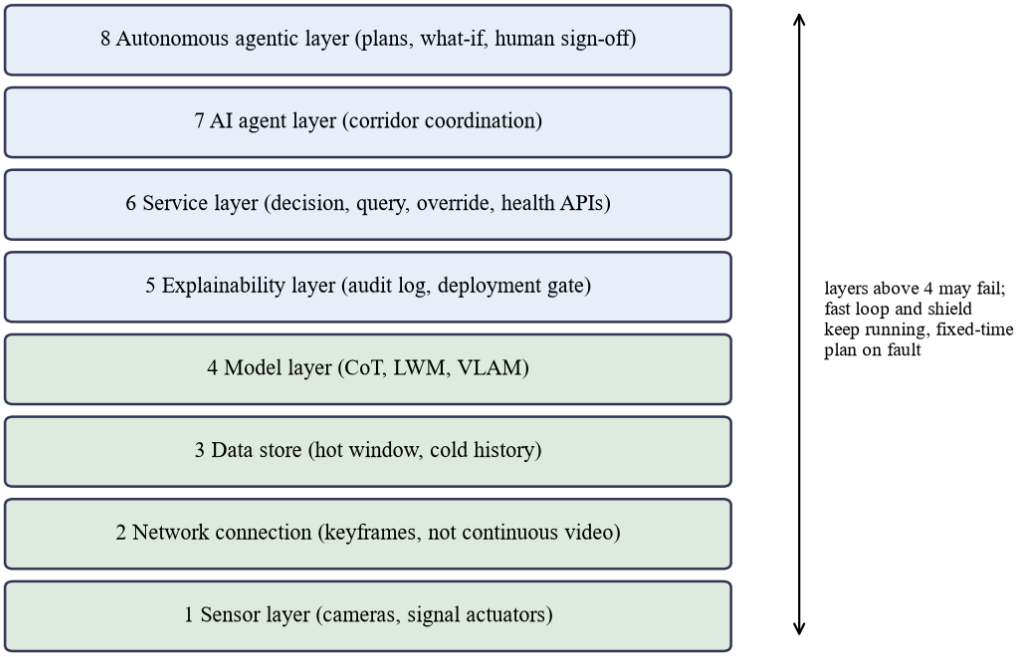


Figure 2**.** The eight-layer deployment framework, credited to Dr. Fasina, with the author's proposed refinements (not yet confirmed by the supervisor)

3.4.2 The cloud versus on-site video tension and its reconciliation

The framework as the supervisor framed it is cloud-based and sends compressed video to the cloud, whereas keeping raw video on site and running the fast loop locally appears to conflict with it. With all processing in the cloud, every signal decision would depend on a network link.

The author proposes the following reconciliation, built on the supervisor's framing and not supervisor-approved. The eight layers are kept. The cloud is the primary tier for the data store, the model layer with the VLAM and its reasoning, the explainability layer, the service layer, the AI agent layer and the autonomous agentic layer. The network layer carries compressed keyframes for the slow loop and not continuous video, and raw video never leaves the site. The sensor layer, the fast loop and the safety shield remain on the edge computer as the resilience tier, so that loss of the network leaves the junction running.

3.5 Cloud deployment architecture

3.5.1 Hosting and placement

The deployed system is intended to be hosted on the cloud, following the supervisor's framework. The cloud is the primary tier for the data store, the model layer, the explainability layer, the service layer, the AI agent layer and the autonomous agentic layer. The junction hosts the sensor layer, edge perception, the fast loop and the safety shield, which form the resilience tier that keeps the signals operating whenever the link weakens or fails. The network layer carries compressed keyframes, summary state, subgoals and log records between them. The cloud region is a design question: a region near Lagos would shorten round trips, a distant one may offer more mature managed services, and the choice interacts with data residency. Components are intended to be packaged as containers, with managed services where available, and no vendor is assumed.

3.5.2 Latency and cost

The slow loop is designed to run every few minutes, so a round trip of several seconds, covering keyframe upload, model inference and return of the subgoal, is tolerable, because the fast loop continues on the subgoal in force while a call is pending. Latency matters for emergency messages and subgoal freshness, not signal timing. Keyframes are single compressed images sent occasionally, which is intended to keep uplink demand small compared with continuous video, although their size and data cost have not been measured. The only latency figure in this work is the wall-clock time of one 3B model call on a four-core local CPU, which is not a cloud figure. No cost has been estimated.

3.5.3 Security and data residency

Every message between an edge node and the cloud is designed to be cryptographically signed, with each edge node authenticated to the cloud and the cloud to each edge node. A message may change priorities and never bypass the shield, so a spoofed message could at worst degrade efficiency and not safety. Components are to run with least privilege: the cloud holds no credentials that can write to the signal controller, which is reached only through the edge computer and the shield. Raw video stays on site.

Data residency is an open question for the supervisor and for the regulators. Whether keyframes, decision logs and reasoning traces may be stored outside Nigeria, and in

which region, has to be settled with LASTMA and the Ministry of Transportation, and the answer may constrain the choice of cloud region.

3.5.4 Behaviour across the degradation tiers

With full connectivity the cloud runs the large VLAM for the slow loop and hosts the upper layers. With weak connectivity the cloud is called less often or not at all: the junction uses a small VLM on the edge or the cached time-of-day schedule, alerts and subgoals may travel by SMS or the junction mesh, and logs wait for store-and-forward. With no connectivity the fast loop continues on the last subgoal and logs are queued locally. On a system fault the junction runs a fixed-time plan, which the cloud layers cannot affect. The prototype's outage scenario simulates only the last two connectivity behaviours, through a scripted window in which the coordinator holds the last subgoal and later switches to a hand-written schedule.

No part of this cloud deployment has been built or tested. The prototype simulates the control logic and the outage behaviour with a mock or scripted backend, plus a single smoke test of a 3B model on a local machine. No measurement of latency, availability, cost, security or data residency exists, and this section states design intent for a staged evaluation described in the final chapter.

3.6 Simulation environment

3.6.1 The junction

The experimental environment is a single signalised T-junction in SUMO. The main road runs east to west with two lanes in each direction at 50 km/h, and the side road is the south arm with one lane in each direction at 40 km/h. Every approach is 300 m long, each arm has a sidewalk and a pedestrian crossing, the step is 1 s, and the sublane model with a lateral resolution of 0.8 m lets motorcycles filter between lanes.

3.6.2 Phases

Three signal phases serve six movements. Phase A serves the main road with the east left turn permissive and pedestrians crossing the south arm. Phase B gives the east approach a protected left with east through and no crossing. Phase C serves the side road, with turns yielding to pedestrians, and gives green to the east and west crossings. A conflict checker confirms that no conflicting movements are green together.

Four vehicle types represent informal transport. A car is 4.5 m long (60 km/h maximum), a danfo is a 5.5 m minibus (50 km/h) that counts as transit, a keke is a 2.7 m tricycle (40 km/h), and an okada is a 2.0 m motorcycle (60 km/h) with arbitrary lateral alignment so that it filters between lanes. On tuning seeds the lateral standard deviation of okada positions ranged from 0.79 m to 0.90 m, against 0.25 m to 0.31 m for cars, which indicates that filtering is present. These are simulation settings, not Lagos measurements.

Each scenario lasts 3600 s of measured time after a 300 s warm-up excluded from every metric, so each run ends at 3900 s. Demand is a seeded Poisson process, so every controller sees the same traffic for a given scenario and seed. The normal scenario represents a typical peak. The market scenario raises pedestrian demand and shifts the mix towards keke and okada. The incident scenario is the normal scenario with the kerb lane of the east approach closed to demand vehicles from 1200 s to 2100 s. The outage scenario carries market demand and makes the slow loop unreachable from 1200 s to 2400 s. Turning shares are 80 and 20 per cent (through and right) from the west, 75 and 25 (through and left) from the east, and 50 each way from the south. The parameters are in the table below.

Table 3**.** Scenario parameters (demand per origin arm in vehicles per hour, vehicle mix, pedestrian demand and events)

Scenario	West, East (veh/h)	South (veh/h)	Mix car/danfo/keke/ okada (%)	Pedestrians (per h)	Event
normal	700, 700	300	45/20/15/20	300	none
market	700, 700	200	30/20/25/25	900	none
incident	700, 700	300	45/20/15/20	300	kerb lane of east approach closed 1200 to 2100 s
outage	700, 700	200	30/20/25/25	900	slow loop unreachable 1200 to 2400 s

Two points matter for interpretation. First, market side-road demand is 200 vehicles per hour and not the 550 specified originally. At 550 the south arm gridlocked under fixed-time control on the tuning seeds, with a mean insertion delay of 1177 s and up to 395 vehicles never inserted. A tuning script tried 550, 300, 275, 250, 225 and 200, requiring at most five uninserted south vehicles on every seed and a mean south insertion delay below 60 s. Only 200 met it, with a mean delay of 3.3 s. Market congestion therefore comes from the vehicle mix, from 900 pedestrians per hour, and from pedestrians blocking the

yielding turns from the south. Second, the incident scenario raises delay only modestly under fixed-time control, because one lane still carries flow.

3.6.5 Seeds and their roles

Three disjoint seed sets serve distinct purposes. Tuning seeds 100 to 104 are used to check scenarios and tune demand. Data seeds 200 to 219 are used with them to collect world-model training data. Evaluation seeds 0 to 4 are used for reported results and never for tuning or training; the collection code refuses them and the loader checks again.

Controllers observe the junction only through a noise model that stands in for an imperfect detector. Each vehicle is missed with probability 0.05, and Gaussian noise with a standard deviation of 10.0 per cent of the value, with a minimum of 0.5, is added to counts, which are then clipped at zero. A clean ground-truth mode is used only for metrics and the audit. The level is an assumption, not a measured detector property.

3.7.1 Baselines that were built

Three baselines were built, all passing through the shield. **fixed_time** cycles through phases A, B and C with greens of 35 s, 12 s and 25 s, plus amber and all-red, representing common pre-timed plans. **count_timer** is a camera-count-plus-timer controller: each green begins at a minimum of 10 s and is extended in steps of 5 s while the served queue is at least 3 vehicles, up to 60 s, with phases cycling A, B, C. **max_pressure** follows the max-pressure principle (Varaiya, 2013), which has been used in learning-based control (Wei et al., 2019a): at each decision it picks the phase whose served movements have the largest total queue, with a fixed green of 15 s. For an isolated junction the downstream queue is treated as empty, a simplification that would not hold in a network.

Two further baselines were planned and are not built: a deep reinforcement learning controller of the kind applied to traffic signals at scale (Wei et al., 2019b; Chu et al., 2020), and a language-model agent such as LLMLight (Lai et al., 2023). The comparison is therefore against classical baselines only.

Three variants are defined. **africot_noslow** uses the fast loop with a fixed default subgoal (all queue weights 1.0, pedestrian weight 1.0, maximum wait 120 s, transit priority 1.0, validity 300 s) and no slow loop. **africot_nowm** has no learned world model and scores candidate first steps with a queue-discharge heuristic from current queues, a smoothed

arrival-rate estimate and fixed discharge rates; its rates are untuned, so it does not bound what a model-free planner could reach. **africot_full** has the slow loop on.

The africot_full variant is implemented and has been wiring-tested with the mock backend on all four scenarios and five seeds, and the real `hf` backend has been called once on one snapshot with the 3B model. It has not been evaluated. A grid of africot_full runs with the 7B model on a GPU is prepared as a notebook for separate execution. Until those runs are available, no statement is made in this dissertation about the effect of the slow loop on traffic performance.

3.8 Metrics

Metrics come from SUMO trip records and per-step sampling, always excluding the warm-up. Average travel time (ATT) and average waiting time (AWT) cover every demand vehicle scheduled to depart after the warm-up and include insertion delay: ATT is departure delay plus trip duration, and AWT is departure delay plus waiting time. A vehicle still in the network at 3900 s, or never inserted, is right-censored at 3900 s, a never-inserted vehicle counting 3900 s minus its scheduled departure. The rule is the same for every controller and was adopted because with SUMO's plain trip means the delay of vehicles queued outside the network vanished, so a controller that starved the side road could look better. The censoring makes AWT a lower bound for vehicles that did not finish.

Average queue length (AQL) is the mean over steps of halting demand vehicles summed over incoming lanes. Throughput is the number of vehicles that arrived, and average pedestrian waiting time is also recorded. Shield interventions, slow-loop calls, parse failures, audit violations and teleports are logged. Each result is a mean and sample standard deviation over the five evaluation seeds.

3.9 The world model and planner

The world model is a small recurrent network in the style of latent world models used for planning, such as DreamerV3 (Hafner et al., 2025) and TD-MPC2 (Hansen et al., 2024), though it is neither of them. An encoder maps the observation to a 32-dimensional latent, a gated recurrent unit takes the latent and the action and gives the next latent, and a decoder predicts the queue per approach, the pedestrians waiting at each crossing and the departures per approach for the next decision. The observation holds per-approach queues, class counts, halting danfos, occupancy and departures, waiting pedestrians per

crossing, movement queues, the phase, time in phase and normalised time of day. The action is the next phase and the green time divided by 60 s. Inputs and targets both come from the noisy perception output, so errors are measured against noisy observations.

Training data are transitions logged from fixed_time, count_timer, max_pressure and a seeded random-timing controller, in 300 episodes from the normal, market and incident scenarios. Outage is omitted because, for controllers without a slow loop, it would duplicate market episodes. The split is by seed: seeds 100 to 104 and 200 to 209 for training (32906 three-decision windows), 210 to 214 for validation (11027) and 215 to 219 for testing (11116). Training uses mean squared error over a three-step rollout, Adam at a learning rate of 0.001, a batch size of 256, a hidden size of 64, early stopping with a patience of 20 epochs, and a single torch seed.

The acceptance check is that, on held-out data, the one-step mean absolute error of queue prediction must beat a persistence guess ("queues stay the same") by at least 10 per cent, with one-step and three-step errors reported. Persistence is a weak comparator, so the threshold is a minimum bar.

The planner uses the Cross-Entropy Method over plans of three steps. Each step has a phase, drawn from a categorical distribution, and a green time, drawn from a Gaussian clipped to 10 s to 60 s and rounded to whole seconds, with an initial mean of 30 s and a standard deviation of 15 s floored at 2 s. Each iteration draws 64 samples, imagines each with the world model, ranks them by cost, and refits the distributions to the 8 lowest-cost samples, for 4 iterations. The cost of an imagined rollout, summed over the horizon, is

$$\text{cost} = \Sigma_a\, w_a \cdot \text{queue}_a + \text{pedestrian_weight} \cdot \text{ped_wait} + \text{transit_priority} \cdot \text{waiting_danfos} + 1000 \cdot \max(0, \text{est_wait}_a - \text{max_wait_s})$$

where w_a are the subgoal's queue weights, and the last term is a heavy penalty on any approach whose estimated wait exceeds the subgoal's maximum wait. The world model does not predict danfos, so the planner approximates waiting danfos as the predicted queue multiplied by the observed danfo share on each approach. The cost is summed per plan step without weighting by step duration, so a long green is not penalised for the time it covers. No planner parameter was tuned.

3.10 Reproducibility and validity

Every reported number is designed to be regenerable from committed code. Each run writes a result file identified by controller, scenario and seed, with its metrics, configuration, git hash and VLAM backend, and mock-backend results are marked not reportable.

The suite of 68 tests passed, and one run per controller on the normal scenario with seed 0 (SUMO 1.27.1) reproduced the committed metrics apart from wall time and git hash. That verified one scenario and seed per controller, not all 100 evaluation runs.

Construct validity is limited because the junction is a plausible Lagos-style layout and not a specific junction, sublane behaviour only approximates informal transport, and perception is ground truth plus assumed noise. Internal validity is supported because all controllers see the same demand and shield and tuning is confined to other seeds, but the world model was trained on four controllers' states and is queried on plans they rarely produce.

Statistical conclusion validity is limited by five evaluation seeds and by large standard deviations in some cells. Paired by-seed comparisons are exploratory, with n equal to 5 and no multiplicity correction, and the exact two-sided Wilcoxon test cannot give a probability below 0.0625 with five seeds. No superiority is claimed where a gap lies within one standard deviation, and such controllers are described as comparable.

External validity is the weakest: one junction, one simulator and no field trial mean the results say nothing about real roads, cameras, a corridor or a cloud deployment.

Five ablations are planned and two have been run: without the slow loop and without the world model, on four scenarios and five seeds. Behaviour under connection loss is represented by the outage scenario, but africot_full is not evaluated. The neighbour-information and routing comparisons are not run, because there is a single junction and no edge-cloud routing.

The table below lists every component of the design, what it specifies, what was built and tested, and its status. "Built and tested" means that the code exists and repository tests exercise it, not that its effect on traffic has been shown.

Table 4. Specified versus built: each component of the design and its status in the prototype

Component	Specified in design	Built and tested in prototype	Status
Edge perception	Edge detector giving queues, counts, occupancy; raw video stays on site	Simulated from SUMO ground truth plus noise; no detector or camera	Partly built (simulated)
Latent world model	Predicts queue evolution; takes neighbour outflows as inputs	GRU latent model (32 dimensions), one junction; one-step error checked against persistence	Built and tested (single junction)
CEM planner (fast loop)	Chooses phase timing, always local	Three-step CEM, 64 samples, 4 iterations, 8 elites, subgoal-weighted cost	Built and tested
Subgoal interface	Compact vector of cost weights and bounds; emergency override separate	Schema with ranges, clamping validator, parser, default subgoal	Built and tested
VLAM with CoT (slow loop)	Reads keyframes and context, outputs subgoal with written justification	Renderer, prompt, `mock` and `hf` backends; one 3B call on one snapshot gave a parseable subgoal	Built; wiring-tested; real-model use limited to one smoke call
Safety shield	Minimum and maximum green, amber, all-red, pedestrian time, spillback rule, starvation guard	All rules except spillback (not applicable at one junction); pedestrian minimum stricter than specified; independent audit	Built and tested (spillback not applicable)
Explainability log	Timestamped log with CoT trace and triggers; proposed frame hash, tamper-evident store	One JSON line per decision; no frame hash or tamper-evident store	Partly built
Capability-aware coordinator	Rule-based routing by connectivity, compute and novelty between edge and cloud	Scheduled and novelty-triggered calls, outage hold and schedule switch, logging; no model routing	Partly built
Degradation tiers	Cloud VLAM; edge VLM or cached schedule; last subgoal; fixed-time plan	Hold-last and cached schedule tested with the mock backend; no edge model; no automatic fixed-time fallback	Partly built
Cloud VLAM tier	Large VLAM in the cloud at full connectivity	Configurable backend (3B on CPU; 7B prepared for GPU); no cloud deployment	Not built as a tier
Cloud hosting of layers 3 to 8	Data store, model, explainability, service, agent and agentic layers hosted on the cloud	None; nothing deployed	Not built
Data residency	Region and storage rules agreed with LASTMA and the Ministry of	None; open question for the supervisor and regulators	Not built; open question

Component	Specified in design	Built and tested in prototype	Status
	Transportation		
Signed-message security	Signed messages, authenticated edge nodes, least privilege	None; no signing or authentication code	Not built
Latency and cost measurement	Round-trip, availability and cost figures for a cloud deployment	None; only one 3B call timed on a local CPU, not a cloud figure	Not measured
Tiered communication (SMS, mesh, satellite, store-and-forward)	Five tiers with signed messages	None; outage modelled as an on-off window for the slow loop	Described only
Emergency override	Verified emergency message forces a phase, subject to clearance	None	Described only
Multi-junction exchange	Neighbour outflows as model inputs; mesh sharing; cluster server	None; single junction	Not built
Eight-layer deployment framework	Cloud-based framework credited to Dr. Fasina, with the author's proposed refinements	A design artefact only; nothing deployed	Described only
Baselines: fixed_time, count_timer, max_pressure	Reference controllers	Built; evaluated on four scenarios and five seeds (max_pressure in isolated-junction form)	Built and evaluated
Baseline: deep RL controller	Reference	Not built	Not built
Baseline: LLMLight	Reference	Not built	Not built
Variants: africot_noslow, africot_nowm	Ablations without the slow loop and without the world model	Built; evaluated on four scenarios and five seeds	Built and evaluated
Variant: africot_full	Full system with slow loop	Implemented; mock-backend wiring tests (20 runs); one 3B smoke call; no evaluation runs	Implemented, wiring-tested, not yet evaluated
Hardware-in-the-loop, shadow mode, pilot	Deployment roadmap	None; the detector test on real Lagos footage is also not done	Described only

The deployment roadmap (Jetson-class edge hardware with hardware-in-the-loop testing, then shadow mode, then a supervised pilot with override, then live operation) is described and not built; cloud hosting and its measurement would be staged within it.

3.12 Chapter summary

The chapter has described a design in which a slow, advisory reasoning loop, intended to be hosted on the cloud, sets priorities, and a fast, local planning loop chooses signal timings under a rule-based shield that keeps the junction running when the link fails. It has also described the much smaller prototype that tests part of the design in simulation. The cloud deployment, routing, tiered communication, emergency override, multi-

junction exchange and the eight-layer framework are described and not built. The next chapter reports what was implemented and what the simulations show.

Chapter 4: Implementation, Results and Discussion

This chapter describes how the prototype specified in the methodology chapter was built, what the committed result files show, and what can be concluded from them. The build proceeded in nine stages, S0 to S8, each ending with passing tests and an independent review whose findings are reported because they changed the evaluation in several places. One point governs everything that follows. The full controller, africot_full, is implemented and wiring-tested with a rule-based mock backend and smoke-tested once with a small vision-language model, but it has not been evaluated. No result in this chapter is a result of the complete system.

4.1 Implementation, stage by stage

4.1.1 Environment and skeleton (S0)

The prototype is written in Python and drives the SUMO traffic simulator headlessly through its TraCI interface (Alvarez Lopez et al., 2018). A pinned requirements file fixes SUMO 1.27.1, a CPU build of PyTorch for the world model, and the transformers library for the vision-language backend. This stage built no controller; it produced the environment, the directory layout and a smoke test that launches SUMO through TraCI, and the reviewer reinstalled the pinned packages and reran the suite without a required fix.

4.1.2 Junction and scenarios (S1)

The second stage built the experimental environment: a signalised T-junction with a two-lane main road in each direction, a single-lane side road to the south, sidewalks, three crossings, and three signal phases as specified. Each phase change passes through an amber of 3 s and an all-red of 2 s. The fleet has four vehicle types (car, danfo, keke and okada), and okadas are configured with arbitrary lateral alignment and a small lateral gap so that they filter between lanes under the sublane model. Across the tuning runs, the lateral standard deviation of okada positions ranged from 0.79 m to 0.90 m, against 0.25 m to 0.31 m for cars, the intended lane filtering. Demand is a seeded Poisson process, so every controller meets the same traffic for a given scenario and seed (see the table below).

Table 5. Scenario parameters as implemented (vehicles per hour per origin arm; pedestrians per hour in total)

Scenario	West, East (veh/h)	South (veh/h)	Mix car/danfo/keke/ okada (%)	Pedestrians (per h)	Event
normal	700, 700	300	45/20/15/20	300	none
market	700, 700	200	30/20/25/25	900	none
incident	700, 700	300	45/20/15/20	300	kerb lane of east approach closed 1200 to 2100 s
outage	700, 700	200	30/20/25/25	900	slow loop unreachable 1200 to 2400 s

Three reviewer-driven changes altered this stage, and each is recorded as a deviation. First, the initial incident, a broken-down vehicle on the kerb lane of the east approach, did not block the lane, because traffic used the lane downstream. The incident is now a closure of that lane to all demand vehicle classes from 1200 s to 2100 s of simulation clock time. Vehicles already on the lane may clear it, and a test confirms that no other demand vehicle enters it during the window.

Second, at the specified side-road demand of 550 vehicles per hour the south arm gridlocked under fixed-time control (mean insertion delay 1177 s, up to 395 south-arm vehicles never inserted on a tuning seed). A tuning script tried six side-road flows on the tuning seeds only, with the criterion of at most five uninserted south-arm vehicles on every seed and a mean south insertion delay below 60 s. Only 200 vehicles per hour met it, (mean south insertion delay 3.3 s, fixed-time mean AWT 45.1 s). The market scenario therefore carries less side-road traffic than the normal scenario (200 against 300 vehicles per hour). Its congestion comes from the vehicle mix and 900 pedestrians per hour. This demand was tuned against fixed-time control (see the threats to validity).

Third, with SUMO's plain trip statistics the delay of vehicles queued outside the network vanishes, so a controller that starved the side road could appear better. Average travel time and average waiting time now include insertion delay, and vehicles still in the network at 3900 s, or never inserted, are right-censored at 3900 s with the same rule for every controller. After these fixes, the fixed-time check on the tuning seeds recorded no jam teleports and at most 4 vehicles never inserted in a run.

4.1.3 Perception, metrics and baselines (S2)

Controllers observe the junction only through a seeded perception model. Per approach it reports the queue, counts by class, halting vehicles (also per movement, because max-

pressure control must distinguish the through phase from the protected-left phase), occupancy and departures, and per crossing it reports waiting pedestrians. Each vehicle or pedestrian is missed with probability 0.05, Gaussian noise with a standard deviation of 10 percent of the value (minimum 0.5) is added, and the result is clipped at zero. Ground-truth fields exist for metrics only, and a test checks that they never reach a controller.

Three baselines were built: a fixed-time plan with greens of 35 s, 12 s and 25 s for phases A, B and C; a count-and-timer controller that starts each green at 10 s and extends it in 5 s steps while the observed served queue is at least 3, up to 60 s; and max-pressure control (Varaiya, 2013), which selects the phase with the largest observed queue on its served movements, treats the downstream as empty because the junction is isolated, and uses a fixed green of 15 s. The stage condition that max-pressure beats fixed-time on waiting time in the normal scenario holds. The first review requested changes, because the market ranking, in which both adaptive baselines appeared worse than fixed-time, was an artefact of metrics that excluded insertion delay; the runs were regenerated after the S1 fixes and the re-review approved the stage. Most result files carry a dirty-tree code hash suffix, but the reviewer verified that the code reproduces them exactly.

4.1.4 Safety shield and its audit (S3)

The shield sits between every controller and the signals and applies its rules to each proposal in a fixed order: a starvation guard that serves next any approach or crossing with observed demand and no green for more than 180 s; a maximum green of 90 s; a minimum green of 10 s for every new phase; and, for any phase that serves a crosswalk, at least the crossing time, which the network gives as 16 s on the east and west crossings and 10 s on the south crossing. Amber and all-red are inserted by the only function that changes the signals, each correction is logged, and the run loop offers no way to disable the shield.

An independent audit class reads simulator ground truth and counts violations of conflicting greens, transition lengths, minimum and maximum green, the pedestrian minimum, and a starvation bound. Thirteen tests exercise the shield and the audit, including deliberately unsafe proposals and a bypassed shield. The audit and the shield share the same rule constants, so the audit shows that the implementation follows the rules, not that the rules suffice for real safety.

One deviation matters for the results below. The specification applies the pedestrian crossing minimum when pedestrians are waiting; the implementation applies it on every phase that gives green to a crosswalk, because perception can miss a waiting pedestrian. In practice this only lifts phase C greens shorter than 16 s. It is stricter than the specification and never less safe, but it makes the correction counts large.

4.1.5 Latent world model (S4)

The world model follows the specification. A multilayer-perceptron encoder maps the noisy perception vector to a 32-dimensional latent, a GRU cell advances the latent given the action (phase and green time), and a decoder predicts, for the next decision, the queue per approach, the waiting pedestrians per crossing and the departures per approach. It supports multi-step rollout and is written in the style of DreamerV3 (Hafner et al., 2025) and TD-MPC2 (Hansen et al., 2024) without being either.

Training data are 300 simulated episodes from four controllers (fixed-time, count-and-timer, max-pressure and a seeded random-timing controller) in the normal, market and incident scenarios. The split is by seed: training uses seeds 100 to 104 and 200 to 209, validation uses 210 to 214 and testing uses 215 to 219, giving 32906, 11027 and 11116 windows. Evaluation seeds 0 to 4 are refused by the collection code. Training used Adam with batch size 256 and early stopping with patience 20, and stopped after 100 epochs. Outage episodes were omitted because they would duplicate market episodes for the data controllers, and inputs and targets both come from noisy perception, so all errors are measured against what the controller observes, not ground truth.

4.1.6 CEM planner (S5)

The planner uses the Cross-Entropy Method (Rubinstein, 1999; de Boer et al., 2005) over three-step plans. Each step is a phase drawn from a categorical distribution and a green time drawn from a Gaussian and clipped to 10 s to 60 s, using 64 samples, 4 iterations and 8 elites with a seeded generator. The cost of an imagined rollout is the weighted predicted queue, plus the pedestrian weight times predicted pedestrian waiting, plus the transit priority times predicted waiting danfos, plus a large penalty for any estimated wait that exceeds the subgoal's maximum wait. Only the first step is executed, after the shield. Because the world model does not predict danfos, the danfo term is approximated by the predicted queue multiplied by the observed danfo share of the queue, an interpretation the

reviewer asked to be stated. The cost is summed per plan step and not weighted by step duration, so a long green is not penalised for the time it covers.

Two variants were run. The africot_noslow variant uses the default subgoal and the trained world model and serves as the without-slow-loop ablation. The africot_nowm variant loads no world model and scores the first step only, using current queues, a smoothed arrival-rate estimate and untuned fixed discharge rates. No planner parameter was tuned on any seed.

4.1.7 Slow loop (S6)

The slow loop has five parts. A renderer draws a 512 by 512 top-down image of the junction from a simulator snapshot, standing in for a camera keyframe. A subgoal module defines the schema and ranges, clamps out-of-range values, and keeps the previous subgoal when a reply cannot be parsed. A VLAM module offers a deterministic mock backend, for tests and wiring only, and a Hugging Face backend defaulting to Qwen2.5-VL-3B-Instruct (Bai et al., 2025) with greedy decoding, configurable to the 7B model for a GPU; the model is asked to reason inside reasoning tags and answer with JSON inside subgoal tags. A coordinator calls the slow loop every 300 simulated seconds or when a novelty trigger fires, holds the last subgoal while the loop is unreachable, and switches to a cached time-of-day schedule after more than 600 s offline. Every decision is written to an explainability log. The reviewer noted that the slow loop is treated as instantaneous in simulated time, so its wall-clock latency has no effect on control, and that the subgoal validity field is unused.

4.1.8 Integration (S7)

The integration stage assembled africot_full, which couples the coordinator, renderer, VLAM and validator to the planner through the subgoal, and checked the wiring with the mock backend on all four scenarios and five seeds. Mock results go to a separate directory flagged as not reportable.

4.1.9 Experiment tooling (S8)

The last stage built a resumable grid runner, the plotting script and a Colab notebook intended to run the africot_full grid with the 7B model on a GPU. The plotting script reads only evaluation-seed runs, drops any mock-backend file, and reports means with the sample standard deviation over five seeds, covering 100 runs (five controllers by four

scenarios by five seeds). The notebook has not been executed, because no GPU was available, so it is untested until its first run.

4.2 Reproduction evidence

The repository's test suite has 68 tests, all passing. During preparation of this dissertation the evidence was checked on a Windows machine with PyTorch 2.12.1 in place of the pinned 2.14.1, which could not be installed there. The full suite passed in 147 s. This reproduction check (gate G3 in the project records) re-ran the normal scenario on seed 0 for fixed-time, count-and-timer, max-pressure, africot_noslow and africot_nowm, and for all five the waiting time, travel time, queue length, throughput, pedestrian waiting time and shield correction counts were identical to the committed files. Only wall time, the recorded code hash and a difference of the order of 1e-8 in the world-model error differed.

The scope is limited to one scenario and one seed per controller. The stage reviewers had separately re-executed a sample of runs with identical metrics, and the world-model reviewer collected the 300 training episodes from scratch and obtained bit-identical weights and metrics.

4.3 World-model accuracy

The world model was evaluated on the held-out test seeds 215 to 219 against a persistence baseline that predicts that queues stay unchanged. The specification required the one-step queue error to beat persistence by at least 10 percent. The model passes this check by a wide margin: the one-step queue mean absolute error (MAE) is 1.60 vehicles against 5.23 for persistence, an improvement of 69.4 percent. The advantage persists at two steps (1.84 against 5.72) and three steps (2.05 against 5.16, 60.3 percent), and one-step errors for waiting pedestrians (0.64 against 1.19) and departures (0.88 against 4.75) show the same pattern.

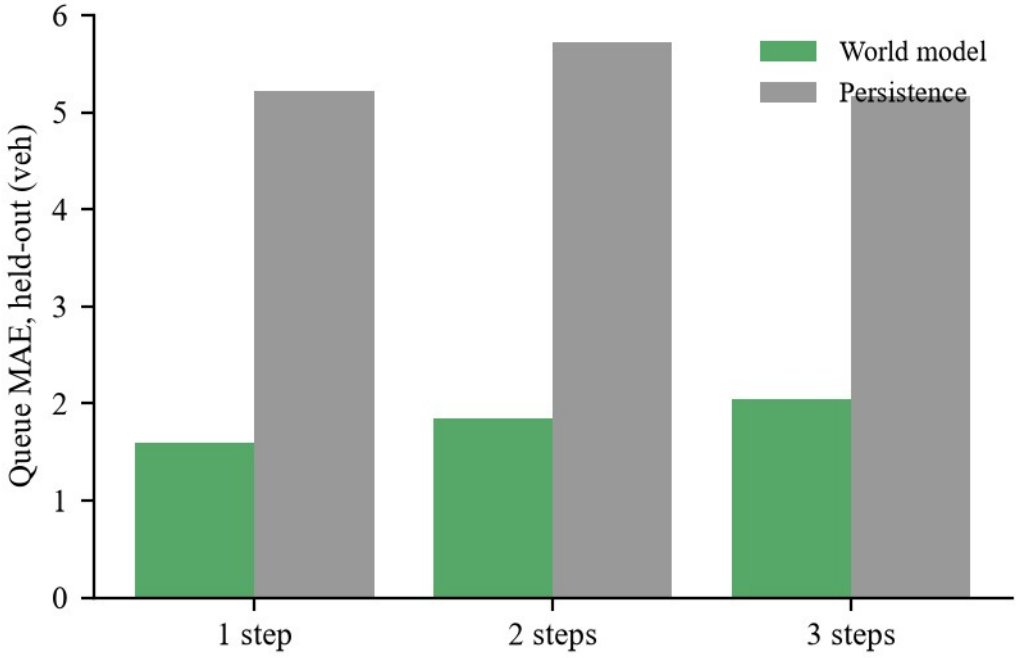


Figure 3. Queue prediction error of the world model and of the persistence baseline on held-out seeds 215 to 219, at one, two and three steps

The figure above shows the model error rising slowly with horizon while persistence stays near five vehicles. Three cautions apply. First, persistence is a weak baseline, because decisions are 15 s to 90 s apart and a green discharges the served queue, so the 10 percent check is a minimum bar and does not show that the model is a strong predictor. Second, both inputs and targets are noisy perception outputs, so the errors are not errors against ground truth. Third, the model was trained once with a single torch seed, so no variability across training runs is available, and it was trained on the state distribution of four controllers whereas the planner queries it on plans none of them would produce.

The online one-step queue error in the africot_noslow evaluation runs has a mean of 0.99 vehicles in the normal scenario, 1.10 in market and 1.50 in the incident scenario, lower than the offline test error, although the incident value is the highest; these too are measured against noisy observations.

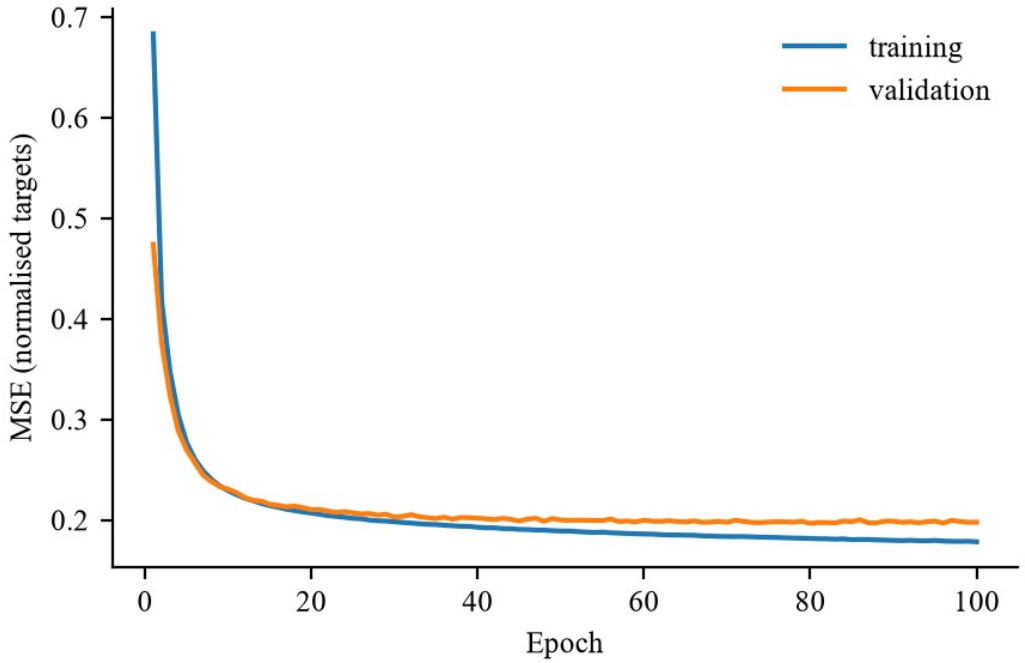


Figure 4. Training and validation loss of the world model by epoch

The training curve above shows the loss used for early stopping, with the best validation loss of 0.197 at epoch 80.

4.4 Controller comparison

4.4.1 Average waiting time

The main comparison covers five controllers in four scenarios, each cell being the mean and sample standard deviation over evaluation seeds 0 to 4. The outage scenario carries market demand and its window only affects the slow loop, so the outage rows of every controller except africot_full equal the market rows; it is not an independent fourth test for these controllers.

Table 6. Average waiting time in seconds, mean and standard deviation over five evaluation seeds

Scenario	fixed_time	count_timer	max_pressure	africot_nowm	africot_noslow
normal	36.3 (10.1)	38.1 (1.3)	17.0 (1.1)	39.9 (10.2)	19.3 (1.5)
market	42.4 (11.1)	44.1 (4.3)	25.9 (13.5)	48.2 (9.1)	23.8 (4.4)
incident	56.8 (6.8)	52.0 (7.4)	35.8 (8.6)	72.6 (29.1)	39.9 (9.2)
outage	42.4 (11.1)	44.1 (4.3)	25.9 (13.5)	48.2 (9.1)	23.8 (4.4)

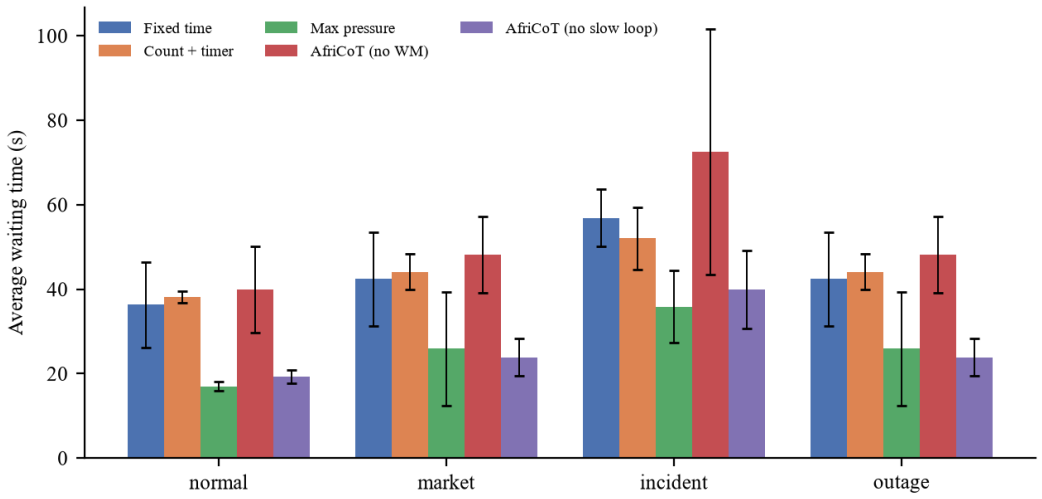


Figure 5. Average waiting time by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group for every controller shown

In the normal scenario, max-pressure has the lowest mean waiting time (17.0 s, standard deviation 1.1 s), followed closely by africot_noslow (19.3 s, 1.5 s). Fixed-time (36.3 s), count-and-timer (38.1 s) and africot_nowm (39.9 s) are about twice as high, and fixed-time and africot_nowm are much more variable. In the market scenario, africot_noslow has the lowest mean (23.8 s, standard deviation 4.4 s) but max-pressure is close (25.9 s) with a standard deviation of 13.5 s, so the lead is inside one standard deviation. In the incident scenario max-pressure again has the lowest mean (35.8 s) and africot_noslow is slightly higher (39.9 s), while africot_nowm is the worst of the five (72.6 s, standard deviation 29.1 s). The incident scenario raises waiting only modestly under fixed-time (from 36.3 s to 56.8 s), because the median east lane still carries flow.

On market, max-pressure is lower than fixed-time on average waiting time in the summary as it now stands (25.9 s against 42.4 s), and count-and-timer is about level with it (44.1 s). An earlier, un-fixed version of the metrics showed both adaptive baselines losing to fixed-time on market, traced by the reviewer to the exclusion of insertion delay and to a gridlocked side road; only the current results are reported.

Table 7. Average waiting time, average travel time, average queue length, throughput and pedestrian waiting time per scenario and controller, mean and standard deviation over five evaluation seeds

Scenario	Controller	AWT (s)	ATT (s)	AQL (veh)	Throughput	Ped. wait (s)	Violations
normal	fixed_time	36.3 (10.1)	100.8 (14.3)	16.8 (4.8)	1698 (26)	19.0 (0.9)	0
normal	count_timer	38.1 (1.3)	104.0 (2.1)	17.8 (0.6)	1704 (17)	13.0 (1.2)	0
normal	max_pressure	17.0 (1.1)	76.5 (1.6)	7.6 (0.6)	1715 (21)	12.5 (0.8)	0
normal	africot_nowm	39.9 (10.2)	104.3 (12.5)	18.2 (4.4)	1702 (15)	19.3 (3.7)	0
normal	africot_noslow	19.3 (1.5)	79.9 (2.3)	8.6 (0.9)	1711 (18)	11.6 (1.1)	0

Scenario	Controller	AWT (s)	ATT (s)	AQL (veh)	Throughput	Ped. wait (s)	Violations
market	fixed_time	42.4 (11.1)	109.4 (15.5)	18.3 (4.8)	1597 (28)	20.3 (0.4)	0
market	count_timer	44.1 (4.3)	111.5 (5.7)	19.2 (2.0)	1601 (20)	14.1 (0.3)	0
market	max_pressure	25.9 (13.5)	88.7 (18.0)	10.7 (5.6)	1598 (31)	15.5 (5.4)	0
market	africot_nowm	48.2 (9.1)	115.9 (12.0)	20.8 (4.2)	1590 (27)	19.4 (1.6)	0
market	africot_noslow	23.8 (4.4)	86.2 (6.5)	9.8 (1.9)	1601 (24)	12.0 (0.8)	0
incident	fixed_time	56.8 (6.8)	132.9 (9.7)	23.0 (3.9)	1697 (20)	18.9 (0.9)	0
incident	count_timer	52.0 (7.4)	125.6 (10.0)	23.7 (2.7)	1702 (14)	16.5 (0.8)	0
incident	max_pressure	35.8 (8.6)	108.3 (10.9)	15.7 (4.1)	1713 (23)	13.8 (1.0)	0
incident	africot_nowm	72.6 (29.1)	147.5 (33.1)	28.5 (6.1)	1690 (10)	22.2 (3.6)	0
incident	africot_noslow	39.9 (9.2)	115.3 (14.1)	16.9 (2.9)	1708 (18)	14.0 (0.9)	0
outage	fixed_time	42.4 (11.1)	109.4 (15.5)	18.3 (4.8)	1597 (28)	20.3 (0.4)	0
outage	count_timer	44.1 (4.3)	111.5 (5.7)	19.2 (2.0)	1601 (20)	14.1 (0.3)	0
outage	max_pressure	25.9 (13.5)	88.7 (18.0)	10.7 (5.6)	1598 (31)	15.5 (5.4)	0
outage	africot_nowm	48.2 (9.1)	115.9 (12.0)	20.8 (4.2)	1590 (27)	19.4 (1.6)	0
outage	africot_noslow	23.8 (4.4)	86.2 (6.5)	9.8 (1.9)	1601 (24)	12.0 (0.8)	0

4.4.2 Other metrics

The other metrics follow the pattern of the waiting time. Throughput differs little between controllers within a scenario: in the normal scenario it ranges only from 1698 to 1715 vehicles, so all controllers serve essentially the same demand. Pedestrian waiting in the normal scenario is lowest for count-and-timer, max-pressure and africot_noslow and highest for fixed-time and africot_nowm.

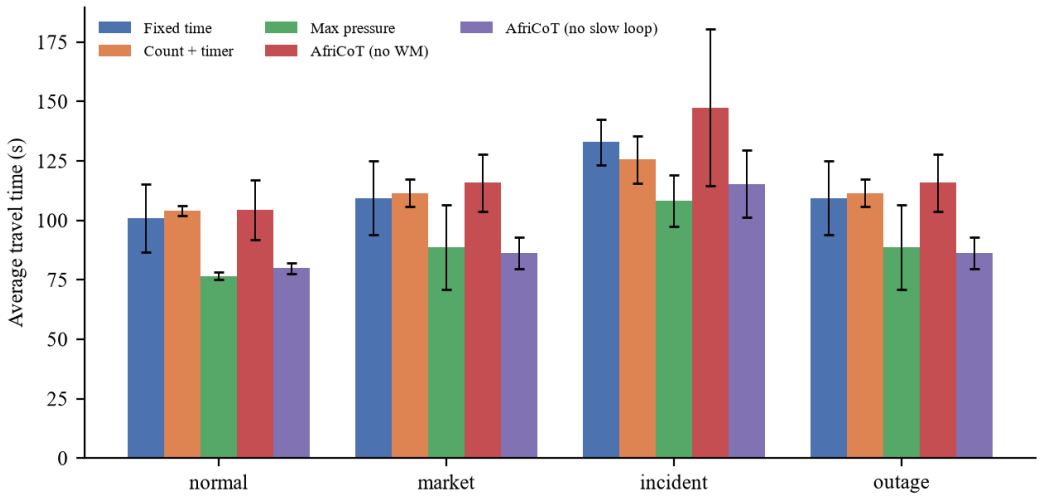


Figure 6. Average travel time by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group

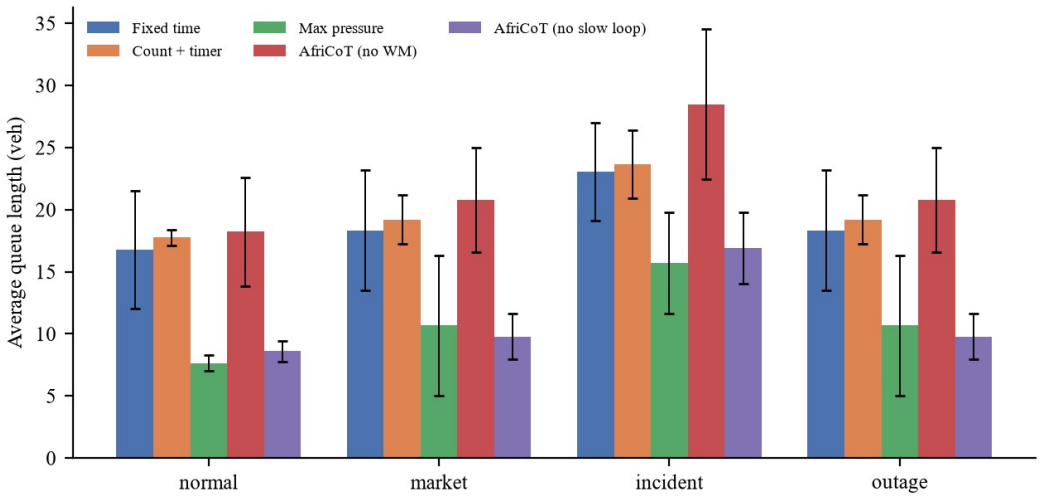


Figure 7. Average queue length by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group

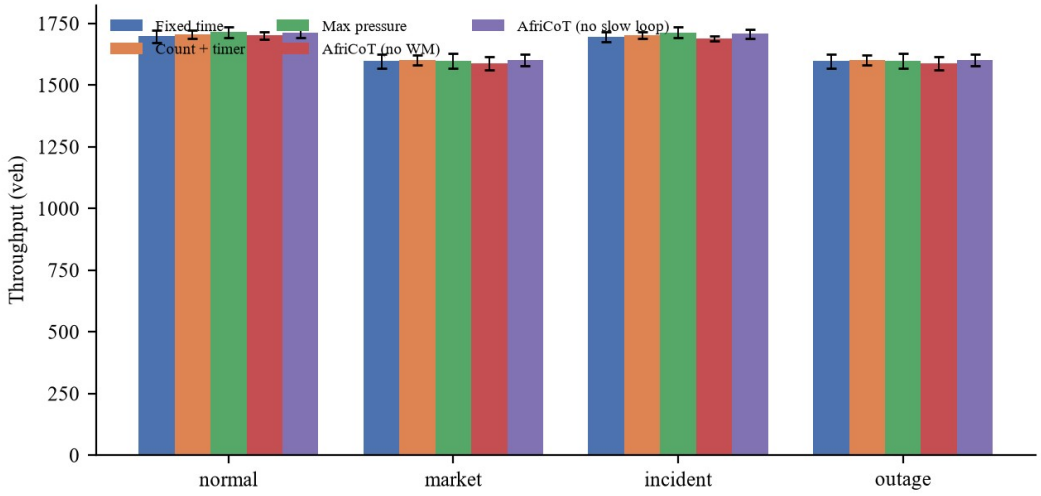


Figure 8. Throughput by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group

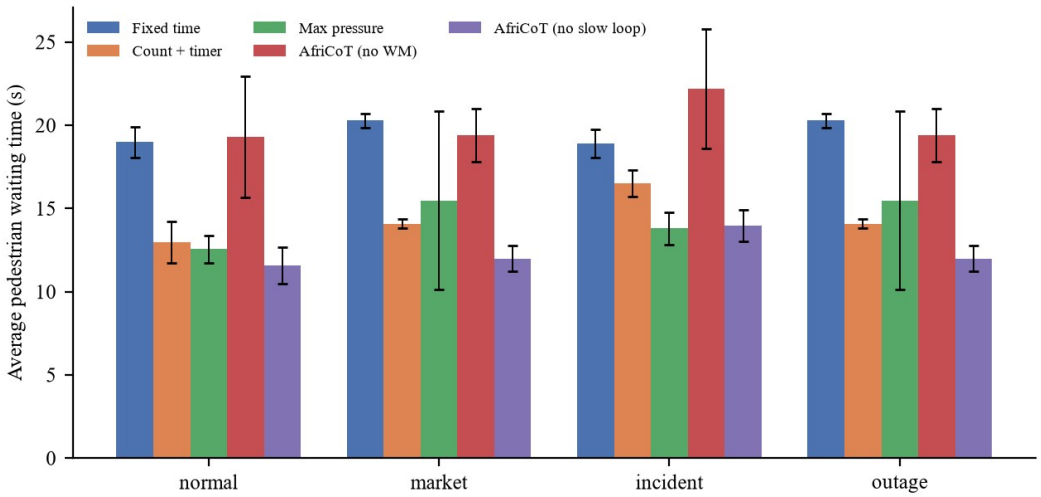


Figure 9. Average pedestrian waiting time by scenario and controller, mean over five evaluation seeds with sample standard deviation as error bars; the outage group duplicates the market group

4.4.3 Paired by-seed tests

Because every controller meets the same traffic for a given seed, differences can be examined seed by seed. The paired tests take the difference in average waiting time between two controllers on each of the five evaluation seeds. They are exploratory: with five seeds the exact two-sided Wilcoxon signed-rank test cannot return a p-value below 0.0625, the t-test rests on five differences, and no correction for multiple comparisons was applied. The count of seeds in which the first-named controller was lower is the more robust summary at this sample size.

Table 8. Paired by-seed differences in average waiting time (first controller minus second), with two-sided p-values; exploratory, five seeds, no multiplicity correction

Scenario	Comparison (A versus B)	Mean AWT difference, A minus B (s)	SD of difference (s)	Paired t-test p	Wilcoxon p	Seeds with A lower
normal	africot_noslow vs max_pressure	2.3	1.1	0.011	0.0625	0 of 5
normal	africot_noslow vs fixed_time	-17.1	9.2	0.015	0.0625	5 of 5
normal	max_pressure vs fixed_time	-19.4	10.1	0.013	0.0625	5 of 5
normal	africot_nowm vs fixed_time	3.6	16.4	0.652	0.4375	1 of 5
normal	africot_noslow vs africot_nowm	-20.6	10.8	0.013	0.0625	5 of 5
market	africot_noslow vs max_pressure	-2.1	15.7	0.780	0.6250	1 of 5
market	africot_noslow vs fixed_time	-18.6	13.4	0.036	0.0625	5 of 5
market	max_pressure	-16.5	8.0	0.010	0.0625	5 of 5

Scenario	Comparison (A versus B)	Mean AWT difference, A minus B (s)	SD of difference (s)	Paired t-test p	Wilcoxon p	Seeds with A lower
market	africot_nowm vs fixed_time	5.8	17.3	0.498	0.8125	2 of 5
market	africot_noslow vs africot_nowm	-24.3	10.6	0.007	0.0625	5 of 5
incident	africot_noslow vs max_pressure	4.1	15.2	0.578	0.4375	1 of 5
incident	africot_noslow vs fixed_time	-16.9	12.2	0.036	0.0625	5 of 5
incident	max_pressure vs fixed_time	-21.0	3.7	0.000	0.0625	5 of 5
incident	africot_nowm vs fixed_time	15.7	30.8	0.317	0.4375	1 of 5
incident	africot_noslow vs africot_nowm	-32.6	23.7	0.037	0.0625	5 of 5

The comparison with max-pressure matters most. In the normal scenario, africot_noslow is higher than max-pressure by 2.3 s on average (standard deviation of the differences 1.1 s), and it was lower in 0 of the five seeds, so max-pressure was lower in all five seeds. In the incident scenario the mean difference is 4.1 s in favour of max-pressure, but the standard deviation of the differences is 15.2 s and africot_noslow was lower in 1 of five seeds, so the two controllers are comparable there (Wilcoxon p = 0.438). In the market scenario the mean difference is -2.1 s with a standard deviation of 15.7 s, africot_noslow was lower in 1 of five seeds, and the p-values (0.78 for the t-test, 0.625 for Wilcoxon) give no basis to separate them. The market and incident results are therefore described as comparable. The paired data do not support a claim that the planner without the slow loop beats max-pressure in any scenario, and they support the opposite on normal demand.

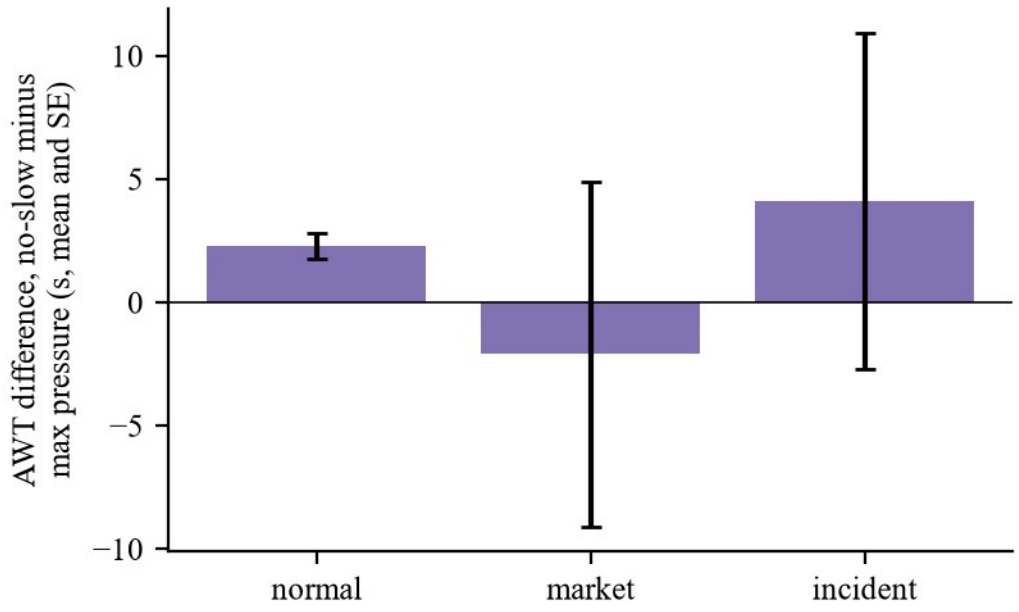


Figure 10**.** Paired difference in average waiting time, africot_noslow minus max-pressure, per scenario, mean over five seeds with standard error as error bars; positive values mean max-pressure is lower

The figure above shows the same pattern.

Against fixed-time, both africot_noslow and max-pressure have lower waiting time in all five seeds in every scenario, with mean differences between -16.5 s and -21.0 s and exact Wilcoxon p-values that sit at the floor of 0.0625. Adaptive control that uses current queues thus has lower waiting time than a fixed plan in these simulations, including the planner with the world model, but this does not separate the planner from max-pressure.

4.5 The contribution of the world model

The ablation without the world model is the clearest result among the variants. In the paired tests, africot_noslow has lower waiting time than africot_nowm in all five seeds in each of the three scenarios, with mean differences of -20.6 s (normal), -24.3 s (market) and -32.6 s (incident), and exact Wilcoxon p-values at the floor of 0.0625 in all three. In the means, africot_nowm is also no better than fixed-time, being higher in every scenario (39.9 s against 36.3 s in normal, 48.2 s against 42.4 s in market and 72.6 s against 56.8 s in incident). The paired tests against fixed-time do not separate them: africot_nowm was lower in 1, 2 and 1 of five seeds in the three scenarios, with small differences relative to their spread. In summary, africot_nowm is comparable to fixed-time and clearly worse than the planner that uses the learned model.

This result needs a caveat. The africot_nowm variant is not a matched ablation of equal strength. It replaces a three-step imagination planner with a one-step queue-discharge heuristic that uses fixed, untuned discharge rates, so the learned predictor and the planning horizon change at once. A world-model-free controller built with comparable care could narrow or remove the gap. The result is evidence that the learned model with three-step planning is better than this particular heuristic, not of how much of the benefit comes from learning as such. On the incident scenario africot_nowm left a mean of 5.4 vehicles uninserted per run, with a waiting-time standard deviation of 29.1 s.

4.6 Safety shield audit

The independent audit recorded 0 safety violations across all 100 evaluation runs. The shield was active in every run and its corrections are summarised below.

Table 9**.** Shield corrections by controller and rule, summed over all evaluation runs of that controller

Controller	Pedestrian crossing	Maximum green	Starvation guard	Total corrections	Safety violations
count_timer	1328	0	0	1328	0
max_pressure	1505	4	2	1511	0
africot_nowm	433	112	0	545	0
africot_noslow	1418	0	0	1418	0

Across all runs the shield made 4684 pedestrian-crossing corrections, 116 maximum-green corrections and 2 starvation corrections.

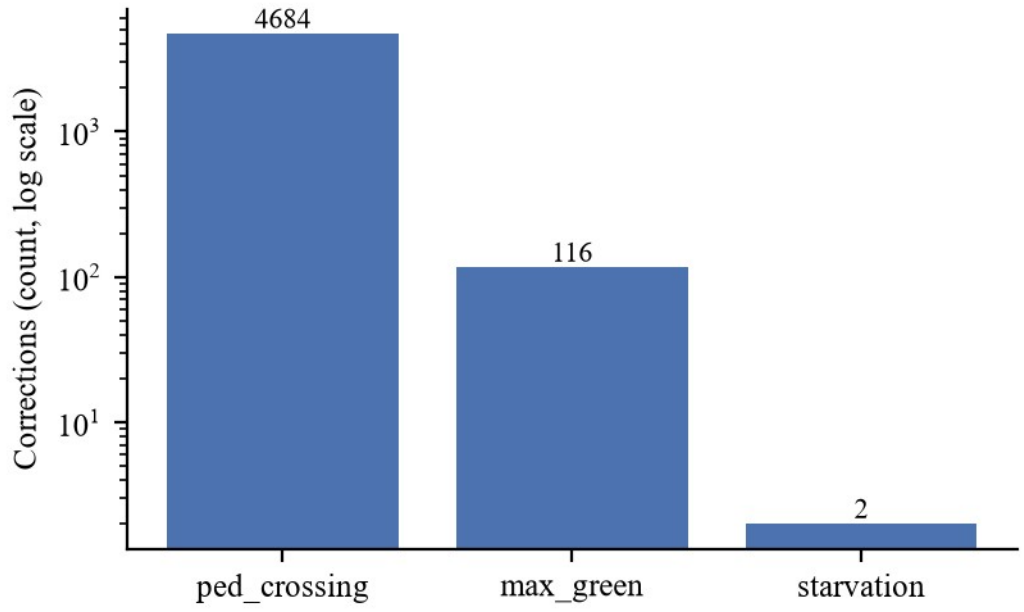


Figure 11**.** Shield corrections by rule summed over all evaluation runs, on a logarithmic scale

Most corrections are the pedestrian crossing minimum, a consequence of the deviation described earlier: any phase C green shorter than 16 s is lifted to that value, so the count reflects how often controllers propose short side-road greens, not frequent unsafe actions. The maximum-green corrections are concentrated in the africot_nowm runs (112 of the 116), where the heuristic repeatedly reselects the same phase, and the two starvation corrections both come from max-pressure. The shield thus acts as a backstop against a weak controller, as intended, and the africot_nowm results are reported after its corrections. A zero audit count concerns the implemented rules, which the audit and shield share, and makes no claim about real-world safety.

Jam teleports, in which SUMO removes a vehicle stuck for too long, occurred in only two cells: 1 in africot_nowm on the normal scenario and 1 each in max-pressure on market and outage, which are the same runs counted twice because outage equals market. The reviewer traced the max-pressure case to vehicles held inside the junction while yielding to dense pedestrian flow, a sublane artefact that the shield cannot reach, and the audit found no signal violation there. Teleports of other kinds were recorded in several cells (up to 5, in africot_nowm on the incident scenario).

4.7 Slow-loop evidence

4.7.1 Wiring checks with the mock backend

All wiring checks used the mock backend, which is rule-based and is not the VLAM. They show that the software paths work and say nothing about reasoning quality or traffic performance. The mock africot_full controller completed all 20 runs with 0 safety violations and 0 parse failures.

In the outage scenario the coordinator log shows the intended degradation sequence on every seed. On seed 0 the state is online at the start, switches to holding the last subgoal at 1200 s, switches to the cached time-of-day schedule at 1807 s, which is more than 600 s into the outage, and returns online at 2413 s. The return to online is logged slightly after the end of the outage window at 2400 s, consistent with connectivity being checked at decision times, although the code was not inspected to confirm it.

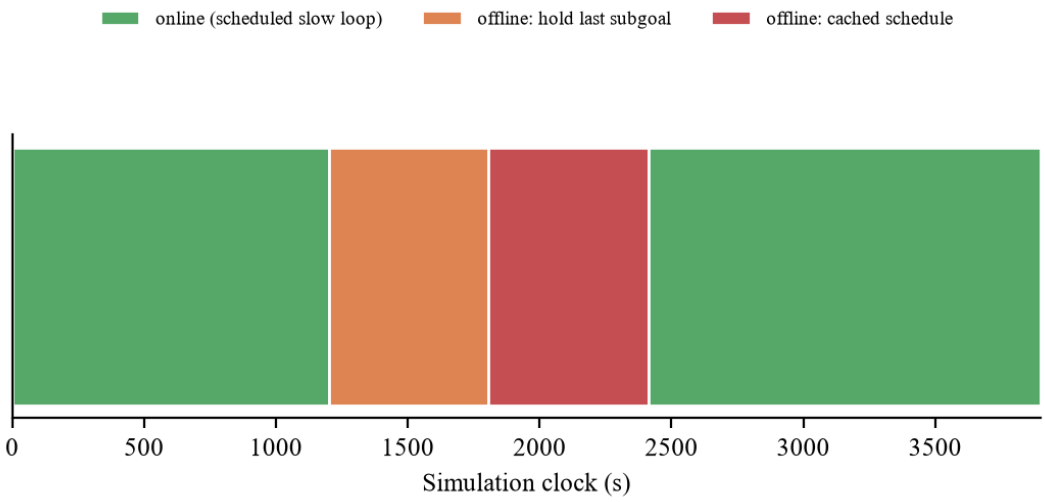


Figure 12**.** Connectivity state of the coordinator over the simulation clock in the outage scenario on seed 0 with the mock backend: online, holding the last subgoal, using the cached schedule, and online again

The novelty trigger fired inside the incident window (1200 s to 2100 s) on all five seeds in the mock runs, which was the stage condition. The trigger is not specific to the incident, however. The same check records novelty events in the normal scenario (seeds 3 and 4), market (seeds 1, 3 and 4) and outage (seed 4), and some incident triggers fall outside the incident window (for example seed 0 at 3350 s and seed 3 at 891 s). Trigger events can also coincide with scheduled calls, so distinct novelty-driven calls are fewer than the event count in some runs. The trigger responds to rises in world-model error generally and cannot be described as an incident detector.

4.7.2 The 3B smoke test

The Hugging Face backend was run once, on the committed renderer snapshot (market scenario, tuning seed 100, 226 s), with Qwen2.5-VL-3B-Instruct on a four-core CPU. The prompt gave a clock time of 07:03, queues of about 0.0, 2.5 and 3.4 on the east, west and south approaches, and about 11 waiting pedestrians at the east crossing against about 2 at the west.

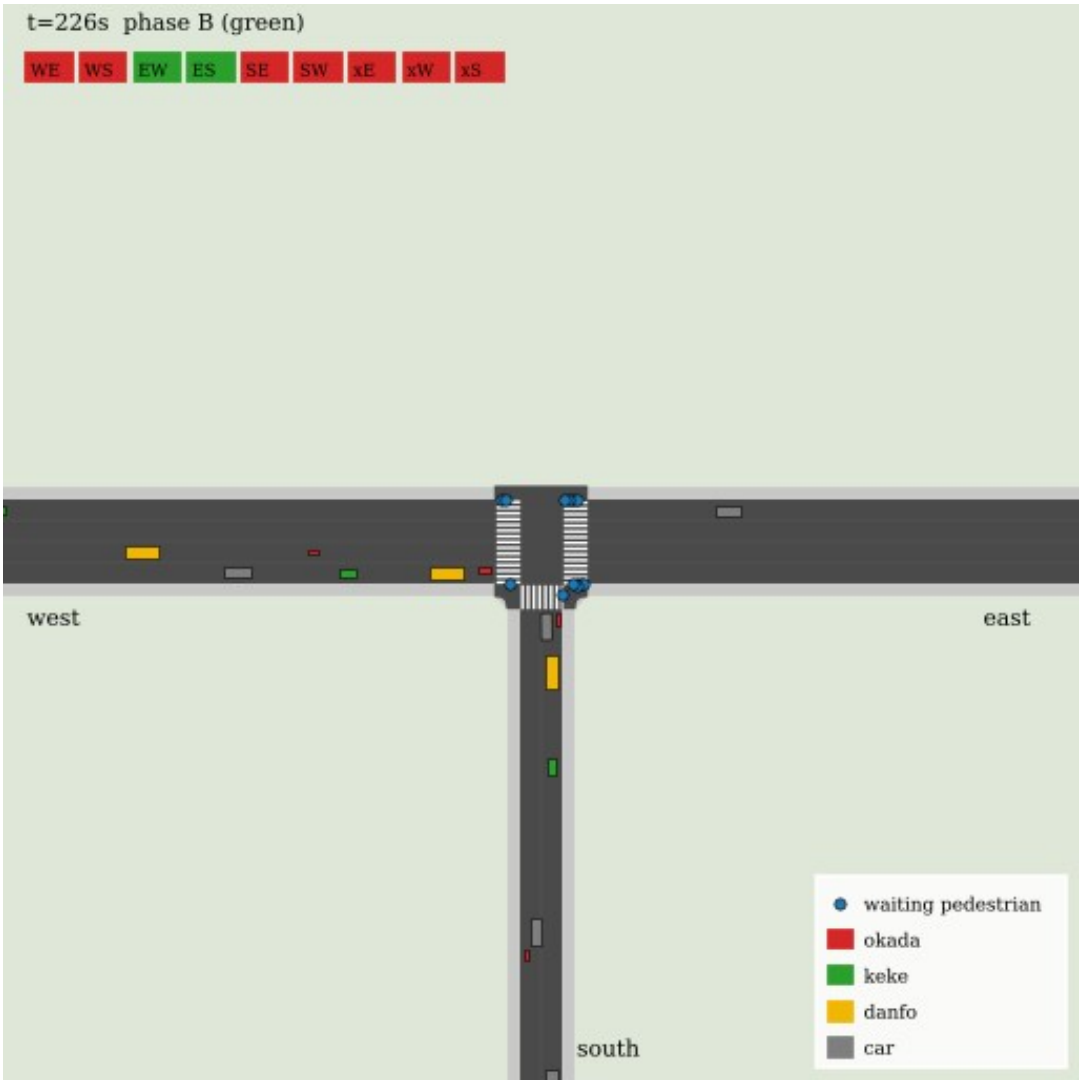


Figure 13**.** Renderer keyframe used for the smoke test: market scenario, tuning seed 100, 226 s into the simulation; waiting pedestrians appear as dots at the crossings, and the signal state per movement is shown at the top left

The reply was parseable and the validator accepted it without notes. The subgoal kept all queue weights at 1.0, raised the pedestrian weight to 3.0, and left the other fields at their defaults. The recorded latency was 146.7 s on an idle machine, about half of the 300 s call interval.

The reasoning text is the more important finding. The model states that 11 pedestrians are waiting on the west arm, but the context gave about 11 at the east crossing and about 2 at the west, so the reasoning is wrong about which arm has the pedestrians. The resulting subgoal (a higher pedestrian weight) is plausible for a market-day scene, but it is not supported by correct reasoning. The call shows that the pipeline from image and context to a validated subgoal works; it is not evidence of sound reasoning and supports no claim about 7B or larger models, which have not been run.

Pending (not yet evaluated). The africot_full grid (slow loop on, Qwen2.5-VL-7B-Instruct, four scenarios, evaluation seeds 0 to 4) is prepared in the Colab notebook and has not been run. No africot_full results are reported in this chapter, and the summary tables contain no africot_full row. When run, this block is to be replaced by the africot_full rows, a paired by-seed comparison with africot_noslow and max-pressure, and a report of parse failures and call latency.

4.8 Negative results and threats to validity

The unfavourable results are stated above and not softened: the planner without the slow loop loses to max-pressure on normal demand (lower in five of five seeds), africot_nowm is no better than fixed-time, the novelty trigger is not incident-specific, the 3B reasoning was wrong, and the slow loop has no evaluation. The main threats to validity follow. First, the evaluation covers a single isolated junction, whereas the design targets a corridor of close junctions, so no result bears on neighbour exchange, green waves or mesh communication. Second, perception is simulator ground truth with seeded noise and missed detections, without occlusion, lighting or class confusion, and no detector was tested on real footage. Third, demand is synthetic Poisson traffic, and the market side-road demand was lowered from 550 to 200 vehicles per hour, tuned against fixed-time control on the tuning seeds. Fourth, there are only five evaluation seeds, so the paired tests are exploratory. Fifth, the outage scenario is not independent of market for any controller without a slow loop. Sixth, africot_nowm is a weaker comparator than a carefully built world-model-free planner. Seventh, the censoring rule makes waiting and travel time a lower bound for unfinished vehicles, applied equally to every controller. Eighth, the planner and baselines were not tuned, so the results may understate tuned versions. Ninth, the slow loop is treated as instantaneous in simulated time, so latency, about 147 s per call for the 3B model on a CPU, has no effect on control in any run. Tenth, no cloud deployment was built, tested or measured: the simulation covers the control logic and the outage behaviour only, and no latency, availability, cost, security or data-residency figure exists for a cloud-hosted system.

The deviations from the specification that are recorded in the project log are summarised in the table below.

Table 10**.** Deviations from the specification recorded during the build

Stage	Deviation	Reason
S1	Incident implemented as closure	The stopped vehicle did not

Stage	Deviation	Reason
	of the east kerb lane to demand classes from 1200 s to 2100 s, replacing a stopped vehicle	block the lane downstream (reviewer finding)
S1	Event times read as simulation clock times, including the warm-up	The specification does not say which clock
S1	Market and outage side-road demand 200 veh/h instead of 550 veh/h	At 550 the south arm gridlocked under fixed-time; 200 passed the tuning criterion
S1, S2	ATT and AWT include insertion delay, with right-censoring at 3900 s	Plain trip statistics hid the delay of vehicles never inserted
S2	Perception also returns halting vehicles per movement	Max-pressure needs per-movement queues
S3	Pedestrian crossing minimum applied on every pedestrian phase	Perception can miss waiting pedestrians; stricter than the specification
S4	World-model data omit the outage scenario; inputs, targets and persistence use noisy perception	Outage duplicates market for the data controllers; ground truth must not be used for control
S5	Danfo cost term approximated from the observed danfo share of the queue	The world model does not predict danfos

4.9 Discussion

The evidence answers some of the research questions and leaves others open.

The first question is whether a small latent world model can learn useful short-horizon dynamics of a junction with mixed informal transit. The evidence says yes in a limited sense: on held-out seeds the one-step queue error is 1.60 vehicles against 5.23 for persistence, rising to 2.05 at three steps. Transfer to real counts is not shown.

The second question is whether planning with the world model beats simpler control, and the answer is mixed. The planner has lower waiting time than fixed-time and than the one-step heuristic variant in all five seeds in each scenario. It is not better than max-pressure in any scenario: it is worse on normal demand by a small and consistent margin and comparable on market and incident, so a simple adaptive rule with a fixed green matches or beats a three-step planner with a learned model here. The planner was not tuned.

The third question is whether the shield keeps every controller within its rules. In simulation the audit recorded no violations in any evaluation run, which supports the structural claim that nothing learned can bypass the shield, not a claim about physical safety.

The fourth question is whether the degradation tiers behave as designed when connectivity is lost. The wiring evidence shows the designed sequence of holding the last subgoal and then switching to the cached schedule after ten minutes. It does not show that the cached schedule is a good one, what degradation costs in traffic terms, or how a cloud deployment would behave.

Several questions cannot be answered. Whether chain-of-thought reasoning improves the subgoals, and through them the traffic outcomes, is untested, because no africot_full result exists, and the single 3B call gives reason for caution about small models. Corridor behaviour, performance on real footage, regulatory acceptance of the explainability log and the behaviour of a cloud-hosted deployment are outside this evaluation.

Pending (not yet evaluated). The central hypothesis of the design, that a slow loop with chain-of-thought reasoning can steer the fast loop through the subgoal to lower delay when conditions are unusual, is intended to be tested by the africot_full grid with the 7B model. Until that grid has been run, this chapter makes no claim that the slow loop helps, hurts or has no effect. This block is to be rewritten with the paired by-seed comparison of africot_full against africot_noslow and max-pressure in all four scenarios, including outage.

Taken together, the results are best read as a working, reproducible prototype with an honest baseline. The fast loop with a learned world model has lower waiting time than fixed-time and than a one-step heuristic in these simulations, does not beat the strongest classical baseline, and the slow loop with chain-of-thought reasoning remains to be evaluated.

Chapter 5: Summary, Conclusion, Recommendations and Future Work

5.1 Summary of the Study

This dissertation set out to examine whether a cloud-based traffic signal controller designed around Sub-Saharan African constraints, with edge fallback for intermittent connectivity, could be specified, built and tested in simulation. The design couples two models only through a compact subgoal. A slow loop, intended to be served by a cloud-hosted vision-language-action model with Chain-of-Thought reasoning, reads keyframes and context and proposes cost weights and constraint bounds. A fast loop uses a latent world model and a Cross-Entropy Method planner to choose phase timings on site, as the resilience tier. A rule-based safety shield sits between the planner and the signals, and every decision is intended to be written to an explainability log. The eight-layer deployment framework, proposed by Dr. Fasina during the class session of 27 July 2026, was kept as the organising structure. The system is intended to be hosted on the cloud, which is the primary tier for the upper layers, with Layer 2 carrying compressed keyframes for the slow loop rather than continuous video, while the fast loop and shield stay on site so the junction keeps operating when the link drops. No cloud deployment has been built or measured: the prototype simulates the control logic and the outage behaviour only. The refinements to individual layers remain the author's proposals and have not been confirmed by the supervisor.

The prototype was built for a single Lagos-style T-junction in SUMO, controlled through TraCI, with four scenarios (normal peak, market day, incident and network outage), five evaluation seeds per cell, and 100 reportable runs in total. Perception is SUMO ground truth with noise, so the detector itself was not tested. Five controllers were evaluated: fixed-time, a camera-count-plus-timer rule, max-pressure, and two ablations of the proposed system, one without the slow loop (`africot_noslow`) and one without the world model (`africot_nowm`). The full controller, `africot_full`, is implemented and wiring-tested with a rule-based mock backend, and the vision-language model was smoke-tested once with Qwen2.5-VL-3B on a CPU, but it has not been evaluated.

The main findings are as follows. The latent world model, trained on data that excluded the evaluation seeds, reached a one-step queue error of 1.60 vehicles on held-out seeds against 5.23 for a persistence baseline. The safety shield recorded 0 violations across all 100 runs, as judged by an independent ground-truth audit. On the normal scenario,

`africot_noslow` reached a mean average waiting time (AWT) of 19.3 s, against 36.3 s for fixed-time but 17.0 s for max-pressure, which was lower in all five seeds. The same ordering holds on the incident scenario (39.9 s against 35.8 s). On the market scenario the two are comparable (23.8 s against 25.9 s, with a standard deviation of 13.5 s for max-pressure), and the paired test is not significant. Removing the world model produced a controller no better than fixed-time: `africot_nowm` had a higher mean AWT than fixed-time in every scenario, for example 39.9 s against 36.3 s on the normal scenario.

5.2 Conclusions

The evidence supports four conclusions, each limited by the simulation-only scope. First, a small latent world model trained on perception-level queue, pedestrian and departure data appears able to predict short-horizon queue evolution on this junction considerably better than assuming no change. This supports using it as the planner's internal model, but it was tested only on one junction and on simulated perception.

Second, the world model appears to matter for the planner, and a model-predictive fast loop is lower in waiting than fixed-time on every scenario in the means. It was not better than max-pressure on the normal and incident scenarios, and the evidence does not show that it is better than max-pressure anywhere. The design is therefore best described as competitive with a strong classical baseline on one scenario and behind it on two. The `africot_nowm` result also shows that the planner by itself is not the source of any benefit: without the world model it was no improvement on fixed-time.

Third, the safety shield behaved as intended. No unsafe timing was recorded in the audited runs, and nearly all corrections concerned pedestrian crossing time (4684 corrections, against 116 for maximum green and 2 for starvation). This supports placing learned or heuristic proposals behind a rule layer that they cannot bypass, though the audit shares the simulator's assumptions about conflicts.

Fourth, the evidence on the slow loop is limited to engineering readiness, and says nothing about cloud hosting. The mock-backend runs showed that scheduled calls, the novelty trigger, the hold-last and cached-schedule fallbacks under outage, and the log fields work as wired. The novelty trigger is not specific to the incident, since it also fired in the normal, market and outage scenarios. The 3B smoke test produced a parseable subgoal, but its reasoning attributed the waiting pedestrians to the wrong arm, so Chain-of-Thought from a model of that size is unreliable. Outage is also not an independent

scenario for any controller except `africot_full`, because the results for the others equal their market results. The outage scenario tests only the fallback logic in simulation, not a real link, and no latency, availability, cost, security or data-residency figure exists for a cloud deployment.

Status of the full controller: `africot_full` has not been evaluated, and no conclusion is drawn about whether the slow loop helps control. The evaluation is intended to be run through the Colab notebook with Qwen2.5-VL-7B-Instruct on a GPU. This paragraph is to be replaced by the outcome, whatever it is, once those runs are committed.

5.3 Contributions

The contributions are modest and should be read as such. The work offers a concrete specification of a two-timescale, coupled (not fused) controller that treats connectivity loss, limited sensing and auditability as design inputs; a reproducible SUMO junction with informal transit classes, motorcycle lane-splitting and four scenarios; a rule-based shield with an independent audit; a world model and planner with ablations; and the full set of results, including those that went against the design. The negative results are part of the contribution, because they indicate where the design needs work before any claim about the slow loop is justified.

5.4 Recommendations

Because the evidence does not show an advantage over max-pressure, practitioners should treat max-pressure behind the shield as the reference controller for any comparable setting, and should regard the present design as a research prototype. Any adaptive controller intended for deployment in Lagos should keep a rule-based shield that no learned component can bypass, a fixed-time fallback, and a decision log suitable for audit by LASTMA and the Ministry of Transportation. Researchers evaluating signal controllers under informal traffic should report censored delay that includes vehicles never inserted, since otherwise a controller that starves a side road can appear better than it is, and should report paired by-seed results rather than relying on means that differ by less than one standard deviation.

5.5 Limitations

The study covers one junction, simulated perception and five seeds per cell, and the market side-road demand had to be lowered from 550 to 200 vehicles per hour to avoid

gridlock. The deep reinforcement learning and LLMLight baselines named in the plan were not built. The paired tests are exploratory, with five seeds and no correction for multiple comparisons. No physical field trial was attempted.

5.6 Future Work

The immediate step is the GPU evaluation of `africot_full` with Qwen2.5-VL-7B-Instruct through the Colab notebook, which writes results and logs to Drive, is resumable, and records the backend and model in every result file. The two missing baselines, a deep reinforcement learning controller and LLMLight, should then be added so that the comparison includes the learned and language-model methods named in the plan.

The next extension is a corridor of two to three junctions with neighbour exchange, in which each world model takes its neighbours' outflows as inputs and queue and plan data are shared, so that the effect of neighbour information and of the mesh relay under outage can be tested directly. Capability-aware routing and the tiered communication scheme should be compared with fixed routing in that setting. The perception module should be tested separately on real Lagos footage, since the present noise model is only a stand-in.

Cloud deployment and its measurement are the largest gap between the design and the evidence. The cloud deployment architecture described in the methodology chapter should be built and measured in staged form: first a cloud-hosted slow loop driven by recorded junction data, then end-to-end tests with the edge node over real links. The measurements needed are the latency of each slow-loop call and its distribution, availability of the cloud tier and the time taken to fall back and recover, the cost of hosting and of keyframe transfer, security testing of message signing, edge authentication and least-privilege access, and the data-residency position, which needs agreement with LASTMA and the Ministry of Transportation. The latency of the 3B model on a CPU is not a cloud figure and should not be reused as one.

The deployment roadmap described in this work, but not built, proceeds in stages: hardware-in-the-loop testing on Jetson-class edge hardware, then shadow mode, in which the system proposes but does not act, then a supervised pilot with operator override, and finally live operation. A timeline cannot yet be given, because the submission and defence date is not yet known. It is to be added once that date is fixed.

5.7 Open Questions for the Supervisor

Several decisions remain with Dr. Fasina: whether the architecture core of the journal paper may be reused in the dissertation; the junction configuration; whether simulation-only evaluation is acceptable; which Lagos junction should be named as the reference; the corridor scope; whether the capability-aware coordinator and tiered communication should be added; how the cloud-based eight-layer framework is to be reconciled with keeping raw video on site and the fast loop local; and where cloud-hosted data may reside. The reconciliation adopted here, with the cloud as the primary tier and the on-site fast loop and shield as the resilience tier, is a proposal pending that confirmation.

References

Adeyemo, O. (2026, June 9). Could AI help Lagos untangle its never-ending traffic jam? Business A.M. https://businessamlive.com/could-ai-help-lagos-untangle-its-never-ending-traffic-jam/

Africa Data Centres Association. (2026). Data centres in Africa 2026: The economic report. https://africadca.org/en/data-centres-in-africa-2026-the-economic-report

Alayrac, J.-B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., Ring, R., Rutherford, E., Cabi, S., Han, T., Gong, Z., Samangooei, S., Monteiro, M., Menick, J., Borgeaud, S., . . . Simonyan, K. (2022). Flamingo: A visual language model for few-shot learning. In Advances in Neural Information Processing Systems (Vol. 35, pp. 23716–23736). Curran Associates. https://doi.org/10.52202/068431-1723

Alvarez Lopez, P., Wiessner, E., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., & Wagner, P. (2018). Microscopic traffic simulation using SUMO. In 2018 21st International Conference on Intelligent Transportation Systems (ITSC) (pp. 2575–2582). IEEE. https://doi.org/10.1109/ITSC.2018.8569938

Ariyo, T. W., & Fasina, E. P. (2026). Towards a unified framework of vision-language-action models and latent world models for low-resource, infrastructure-constrained environments in Sub-Saharan Africa [Manuscript submitted for publication]. Machine Learning Research Group, Department of Artificial Intelligence and Robotics, University of Lagos.

Assran, M., Bardes, A., Fan, D., Garrido, Q., Howes, R., Komeili, M., Muckley, M., Rizvi, A., Roberts, C., Sinha, K., Zholus, A., Arnaud, S., Gejji, A., Martin, A., Hogan, F. R., Dugas, D., Bojanowski, P., Khalidov, V., Labatut, P., . . . Ballas, N. (2025). V-JEPA 2: Self-supervised video models enable understanding, prediction and planning (arXiv:2506.09985). arXiv. https://doi.org/10.48550/arXiv.2506.09985

Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., Zhong, H., Zhu, Y., Yang, M., Li, Z., Wan, J., Wang, P., Ding, W., Fu, Z., Xu, Y., . . . Lin, J. (2025). Qwen2.5-VL technical report (arXiv:2502.13923). arXiv. https://doi.org/10.48550/arXiv.2502.13923

Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., . . . Zhilinsky, U. (2025). π0: A vision-language-action flow model for general robot control. In Robotics: Science and Systems XXI. https://doi.org/10.15607/RSS.2025.XXI.010

Black, K., Nakamoto, M., Atreya, P., Walke, H., Finn, C., Kumar, A., & Levine, S. (2024). Zero-shot robotic manipulation with pre-trained image-editing diffusion

models. In The Twelfth International Conference on Learning Representations (ICLR 2024). https://doi.org/10.48550/arXiv.2310.10639

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J. Q., Demszky, D., . . . Liang, P. (2021). On the opportunities and risks of foundation models (arXiv:2108.07258). arXiv. https://doi.org/10.48550/arXiv.2108.07258

Chu, T., Wang, J., Codecà, L., & Li, Z. (2020). Multi-agent deep reinforcement learning for large-scale traffic signal control. IEEE Transactions on Intelligent Transportation Systems, 21(3), 1086–1095. https://doi.org/10.1109/TITS.2019.2901791

Chua, K., Calandra, R., McAllister, R., & Levine, S. (2018). Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In Advances in Neural Information Processing Systems (Vol. 31). Curran Associates. https://doi.org/10.48550/arXiv.1805.12114

de Boer, P.-T., Kroese, D. P., Mannor, S., & Rubinstein, R. Y. (2005). A tutorial on the cross-entropy method. Annals of Operations Research, 134(1), 19–67. https://doi.org/10.1007/s10479-005-5724-z

Driess, D., Xia, F., Sajjadi, M. S. M., Lynch, C., Chowdhery, A., Ichter, B., Wahid, A., Tompson, J., Vuong, Q., Yu, T., Huang, W., Chebotar, Y., Sermanet, P., Duckworth, D., Levine, S., Vanhoucke, V., Hausman, K., Toussaint, M., Greff, K., . . . Florence, P. (2023). PaLM-E: An embodied multimodal language model. In Proceedings of the 40th International Conference on Machine Learning (Vol. 202, pp. 8469–8488). PMLR. https://doi.org/10.48550/arXiv.2303.03378

Du, Y., Yang, M., Dai, B., Dai, H., Nachum, O., Tenenbaum, J. B., Schuurmans, D., & Abbeel, P. (2023). Learning universal policies via text-guided video generation. In Advances in Neural Information Processing Systems (Vol. 36). https://doi.org/10.48550/arXiv.2302.00111

Feng, T., Wang, X., Jiang, Y.-G., & Zhu, W. (2025). Embodied AI: From LLMs to world models. IEEE Circuits and Systems Magazine, 25(4), 14–37. https://doi.org/10.1109/MCAS.2025.3603693

Figure AI. (2025, February 20). Helix: A vision-language-action model for generalist humanoid control [Technical report]. https://www.figure.ai/news/helix

GSMA. (2024a). The mobile economy Sub-Saharan Africa 2024. GSMA Intelligence. https://www.gsma.com/solutions-and-impact/connectivity-for-good/mobile-economy/wp-content/uploads/2024/11/GSMA_ME_SSA_2024_Web.pdf

GSMA. (2024b). The state of mobile internet connectivity report 2024. https://gsma.com/r/wp-content/uploads/2024/10/The-State-of-Mobile-Internet-Connectivity-Report-2024.pdf

Gupta, T., Gong, W., Ma, C., Pawlowski, N., Hilmkil, A., Scetbon, M., Rigter, M., Famoti, A., Llorens, A. J., Gao, J., Bauer, S., Kragic, D., Schölkopf, B., & Zhang, C. (2024). The essential role of causality in foundation world models for embodied AI (arXiv:2402.06665). arXiv. https://doi.org/10.48550/arXiv.2402.06665

Hafner, D., Lee, K.-H., Fischer, I., & Abbeel, P. (2022). Deep hierarchical planning from pixels. In Advances in Neural Information Processing Systems (Vol. 35). https://doi.org/10.48550/arXiv.2206.04114

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning latent dynamics for planning from pixels. In Proceedings of the 36th International Conference on Machine Learning (Vol. 97, pp. 2555–2565). PMLR. https://doi.org/10.48550/arXiv.1811.04551

Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Mastering diverse control tasks through world models. Nature, 640(8059), 647–653. https://doi.org/10.1038/s41586-025-08744-2

Hansen, N., Su, H., & Wang, X. (2024). TD-MPC2: Scalable, robust world models for continuous control. In The Twelfth International Conference on Learning Representations (ICLR 2024). https://doi.org/10.48550/arXiv.2310.16828

International Energy Agency, International Renewable Energy Agency, United Nations Statistics Division, World Bank, & World Health Organization. (2025). Tracking SDG 7: The energy progress report 2025. World Bank. https://www.iea.org/reports/tracking-sdg7-the-energy-progress-report-2025

ITEdgeNews. (2026, April 29). LAMATA deploys AI-powered smart traffic systems to cut congestion on Lagos roads. https://www.itedgenews.africa/lamata-deploys-ai-powered-smart-traffic-systems-to-cut-congestion-on-lagos-roads/

Jiang, Y., Gupta, A., Zhang, Z., Wang, G., Dou, Y., Chen, Y., Fei-Fei, L., Anandkumar, A., Zhu, Y., & Fan, L. (2023). VIMA: Robot manipulation with multimodal prompts. In Proceedings of the 40th International Conference on Machine Learning (Vol. 202, pp. 14975–15022). PMLR. https://doi.org/10.48550/arXiv.2210.03094

Kenya Urban Roads Authority. (2023, February 28). Intelligent transport systems. https://kura.go.ke/news/intelligent-transport-systems/13363/

Kim, M. J., Finn, C., & Liang, P. (2025). Fine-tuning vision-language-action models: Optimizing speed and success. In Robotics: Science and Systems XXI. https://doi.org/10.15607/RSS.2025.XXI.017

Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E. P., Sanketi, P. R., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., & Finn, C. (2024). OpenVLA: An open-source vision-language-action model. In Proceedings of the 8th Conference on Robot

Learning (Vol. 270, pp. 2679–2713). PMLR. https://doi.org/10.48550/arXiv.2406.09246

Kinyanjui, M. (2024, November 27). Phase one of Sh7.18bn Nairobi Intelligent Traffic System to kick off in March. The Eastleigh Voice. https://eastleighvoice.co.ke/city-affairs/92971/phase-one-of-sh7.18bn-nairobi-intelligent-traffic-system-to-kick-off-in-march

Lai, S., Xu, Z., Zhang, W., Liu, H., & Xiong, H. (2023). LLMLight: Large language models as traffic signal control agents (arXiv:2312.16044). arXiv. https://doi.org/10.48550/arXiv.2312.16044

Li, H., Chen, Y., Cui, W., Liu, W., Liu, K., Zhou, M., Zhang, Z., & Zhao, D. (2025). Survey of vision-language-action models for embodied manipulation (arXiv:2508.15201) [Preprint, in Chinese]. arXiv. https://doi.org/10.48550/arXiv.2508.15201

Lin, J., Du, Y., Watkins, O., Hafner, D., Abbeel, P., Klein, D., & Dragan, A. (2024). Learning to model the world with language. In Proceedings of the 41st International Conference on Machine Learning (Vol. 235, pp. 29992–30017). PMLR. https://doi.org/10.48550/arXiv.2308.01399

Liu, S., Tang, J., Hassan, M. S., Sharkawy, M. H., Fouda, M. M. G., Shang, T., & Wang, Z. (2026). Infrastructure first: Enabling embodied AI for science in the Global South (arXiv:2604.06722). arXiv. https://doi.org/10.48550/arXiv.2604.06722

Mumbi, L. (2026, May 12). Government to roll out AI-powered traffic lights at 125 Nairobi junctions. The Eastleigh Voice. https://eastleighvoice.co.ke/city-affairs/348540/government-to-roll-out-ai-powered-traffic-lights-at-125-nairobi-junctions

Nairobi Law Monthly. (2026, June 17). Govt opens Sh10.8bn tender for Nairobi smart traffic overhaul. https://nairobilawmonthly.com/govt-opens-sh10-8bn-tender-for-nairobi-smart-traffic-overhaul/

NVIDIA, Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., Jang, J., Jiang, Z., Kautz, J., Kundalia, K., Lao, L., Li, Z., Lin, Z., Lin, K., . . . Zhu, Y. (2025). GR00T N1: An open foundation model for generalist humanoid robots (arXiv:2503.14734). arXiv. https://doi.org/10.48550/arXiv.2503.14734

Octo Model Team, Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., Luo, J., Tan, Y. L., Chen, L. Y., Sanketi, P., Vuong, Q., Xiao, T., Sadigh, D., Finn, C., & Levine, S. (2024). Octo: An open-source generalist robot policy. In Robotics: Science and Systems XX. https://doi.org/10.15607/RSS.2024.XX.090

Olomu, S. (2025, February 7). Lagos picks Huawei to deploy intelligent traffic system. ITWeb Africa. https://itweb.africa/content/JBwEr7n3ngKM6Db2

Open X-Embodiment Collaboration. (2024). Open X-Embodiment: Robotic learning datasets and RT-X models. In 2024 IEEE International Conference on Robotics and Automation (ICRA) (pp. 6892–6903). IEEE. https://doi.org/10.1109/ICRA57147.2024.10611477

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In Proceedings of the 38th International Conference on Machine Learning (Vol. 139, pp. 8748–8763). PMLR. https://doi.org/10.48550/arXiv.2103.00020

Reed, S., Zolna, K., Parisotto, E., Gómez Colmenarejo, S., Novikov, A., Barth-Maron, G., Giménez, M., Sulsky, Y., Kay, J., Springenberg, J. T., Eccles, T., Bruce, J., Razavi, A., Edwards, A., Heess, N., Chen, Y., Hadsell, R., Vinyals, O., Bordbar, M., & de Freitas, N. (2022). A generalist agent. Transactions on Machine Learning Research. https://doi.org/10.48550/arXiv.2205.06175

Rubinstein, R. (1999). The cross-entropy method for combinatorial and continuous optimization. Methodology and Computing in Applied Probability, 1(2), 127–190. https://doi.org/10.1023/A:1010091220143

Sapkota, R., Cao, Y., Roumeliotis, K. I., & Karkee, M. (2025). Vision-language-action (VLA) models: Concepts, progress, applications and challenges (arXiv:2505.04769). arXiv. https://doi.org/10.48550/arXiv.2505.04769

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., Lillicrap, T., & Silver, D. (2020). Mastering Atari, Go, chess and shogi by planning with a learned model. Nature, 588(7839), 604–609. https://doi.org/10.1038/s41586-020-03051-4

Shi, L. X., Ichter, B., Equi, M., Ke, L., Pertsch, K., Vuong, Q., Tanner, J., Walling, A., Wang, H., Fusai, N., Li-Bell, A., Driess, D., Groom, L., Levine, S., & Finn, C. (2025). Hi Robot: Open-ended instruction following with hierarchical vision-language-action models. In Proceedings of the 42nd International Conference on Machine Learning (Vol. 267). PMLR. https://doi.org/10.48550/arXiv.2502.19417

United Nations Statistics Division. (2025). The Sustainable Development Goals report 2025: Extended report, Goal 7. United Nations. https://unstats.un.org/sdgs/report/2025/extended-report/Extended-Report-2025_Goal-7.pdf

Varaiya, P. (2013). Max pressure control of a network of signalized intersections. Transportation Research Part C: Emerging Technologies, 36, 177–195. https://doi.org/10.1016/j.trc.2013.08.014

Wei, H., Chen, C., Zheng, G., Wu, K., Gayah, V., Xu, K., & Li, Z. (2019a). PressLight: Learning max pressure control to coordinate traffic signals in arterial network. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge

Discovery & Data Mining (pp. 1290–1298). ACM. https://doi.org/10.1145/3292500.3330949

Wei, H., Xu, N., Zhang, H., Zheng, G., Zang, X., Chen, C., Zhang, W., Zhu, Y., Xu, K., & Li, Z. (2019b). CoLight: Learning network-level cooperation for traffic signal control. In Proceedings of the 28th ACM International Conference on Information and Knowledge Management (pp. 1913–1922). ACM. https://doi.org/10.1145/3357384.3357902

Wei, H., Zheng, G., Gayah, V., & Li, Z. (2019c). A survey on traffic signal control methods (arXiv:1904.08117). arXiv. https://doi.org/10.48550/arXiv.1904.08117

World Bank. (2026). Access to electricity (% of population) (EG.ELC.ACCS.ZS) [Data set]. World Development Indicators. Retrieved October 15, 2026, from https://data.worldbank.org/indicator/EG.ELC.ACCS.ZS

Wu, P., Escontrela, A., Hafner, D., Abbeel, P., & Goldberg, K. (2022). DayDreamer: World models for physical robot learning. In Proceedings of the 6th Conference on Robot Learning (Vol. 205, pp. 2226–2240). PMLR. https://doi.org/10.48550/arXiv.2206.14176

Xu, Y. (2025). Opinion: A unified world model is the cornerstone for integrating perception, reasoning, and decision-making in embodied AI [Poster presentation]. NeurIPS 2025 Workshop on Embodied World Models for Decision Making. https://openreview.net/forum?id=WjmfSw2eg9

Yang, S., Li, H., Wang, B., Chen, Y., Tian, Y., Wang, T., Wang, H., Zhao, F., Liao, Y., & Pang, J. (2025). InstructVLA: Vision-language-action instruction tuning from understanding to manipulation (arXiv:2507.17520). arXiv. https://doi.org/10.48550/arXiv.2507.17520

Yin, S., Pang, X., Ding, Y., Chen, M., Bi, Y., Xiong, Y., Huang, W., Xiang, Z., Shao, J., & Chen, S. (2024). SafeAgentBench: A benchmark for safe task planning of embodied LLM agents (arXiv:2412.13178). arXiv. https://doi.org/10.48550/arXiv.2412.13178

Zawalski, M., Chen, W., Pertsch, K., Mees, O., Finn, C., & Levine, S. (2024). Robotic control via embodied chain-of-thought reasoning. In Proceedings of the 8th Conference on Robot Learning (Vol. 270, pp. 3157–3181). PMLR. https://doi.org/10.48550/arXiv.2407.08693

Zhang, W., Kong, X., Bräunl, T., & Hong, J. B. (2024). SafeEmbodAI: A safety framework for mobile robots in embodied AI systems (arXiv:2409.01630). arXiv. https://doi.org/10.48550/arXiv.2409.01630

Zhen, H., Qiu, X., Chen, P., Yang, J., Yan, X., Du, Y., Hong, Y., & Gan, C. (2024). 3D-VLA: A 3D vision-language-action generative world model. In Proceedings of the 41st International Conference on Machine Learning (Vol. 235, pp. 61229–61245). PMLR. https://doi.org/10.48550/arXiv.2403.09631

Zhou, S., Du, Y., Chen, J., Li, Y., Yeung, D.-Y., & Gan, C. (2024). RoboDreamer: Learning compositional world models for robot imagination. In Proceedings of the 41st International Conference on Machine Learning (Vol. 235). PMLR. https://doi.org/10.48550/arXiv.2404.12377

Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohlhart, P., Welker, S., Wahid, A., Vuong, Q., Vanhoucke, V., Tran, H., Soricut, R., Singh, A., Singh, J., Sermanet, P., Sanketi, P. R., Salazar, G., . . . Han, K. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. In Proceedings of the 7th Conference on Robot Learning (Vol. 229, pp. 2165–2183). PMLR. https://doi.org/10.48550/arXiv.2307.15818

Appendix A: Reproduction Commands

All commands are run from the root of the prototype repository. The code is designed so that every reported number can be regenerated from committed files, and an independent check at the third gate reproduced the normal-scenario, seed 0 results of the five non-VLAM controllers exactly, apart from wall time, git hash and a float difference of the order of 1e-8 in the world-model error. Install and test with the same interpreter, for example through `python -m pip`.

A.1 Environment. SUMO is installed from the pinned pip packages, so no separate system installation is required. The requirements file pins eclipse-sumo, traci and sumolib at version 1.27.1, numpy 2.4.6, matplotlib 3.11.2, pytest 9.1.1, torch 2.14.1 (CPU wheel index), torchvision 0.29.1, transformers 5.19.0, accelerate 1.15.0, pillow 12.3.0 and qwen-vl-utils 0.0.14. The helper in the package locates the SUMO binaries and sets `SUMO_HOME`; the graphical interface is never used.

- `python -m pip install -r requirements.txt`
- `sh sim/build_net.sh` (rebuilds `sim/junction.net.xml` with netconvert; the committed network is already built)

A.2 Tests. The test suite passed with 68 tests at the start of the documentation work and takes roughly 147 s.

- `python -m pytest -q`

A.3 Single runs and grids. One controller on one scenario and seed writes `results/<controller>__<scenario>__<seed>.json`; existing results are skipped unless `--force` is given. The scenarios are normal, market, incident and outage; the controllers are fixed_time, count_timer, max_pressure, africot_noslow, africot_nowm and africot_full. The default VLAM backend is the mock, whose runs are written for wiring checks only and are never reported.

- `python run_experiment.py --controller max_pressure --scenario normal --seed 0`
- `python run_grid.py --controllers fixed_time count_timer max_pressure --seeds 0 1 2 3 4 --jobs 4`
- `python run_grid.py --controllers africot_noslow africot_nowm --seeds 0 1 2 3 4 --jobs 4`

A.4 World model. Data are collected on tuning seeds 100 to 104 and data seeds 200 to 219, never on evaluation seeds 0 to 4, and the model is then trained.

- `python train_world_model.py collect --jobs 4`
- `python train_world_model.py train`

A.5 Tables, figures and facts. The plotting script uses only reportable runs on evaluation seeds 0 to 4 and writes `results/summary.csv`, `results/summary.md` and the figures. The facts file from which every number in this document is filled is built from committed files only.

- `python plots.py`
- `python scripts/build_facts.py`

A.6 The africot_full evaluation on a GPU (Colab). This step has not yet been executed. The notebook `colab_experiments.ipynb` mounts Google Drive, clones the build branch, installs the requirements while keeping Colab's own CUDA build of PyTorch, runs two test files, and then runs the resumable grid with the Qwen2.5-VL-7B-Instruct backend. The 7B model in bfloat16 needs about 17 GB of GPU memory. The grid cell is:

- `python run_grid.py --controllers africot_full --scenarios normal market incident outage --seeds 0 1 2 3 4 --vlam-backend hf --vlam-model Qwen/Qwen2.5-VL-7B-Instruct --results-dir <Drive results folder> --jobs 1`

The notebook then prints the backend, model, slow-loop calls, parse failures and mean latency per run, and the final cell copies the result files into `results/` and reruns `python plots.py`. The smoke test of the 3B model on a CPU used the same hf backend and took 147 s for one call.

Appendix B: Scenario and Shield Parameters

Table 11**.** Scenario parameters (demand in vehicles per hour, pedestrians per hour, vehicle mix and event windows)

Scenario	West, East (veh/h)	South (veh/h)	Mix car/danfo/keke/ okada (%)	Pedestrians (per h)	Event
normal	700, 700	300	45/20/15/20	300	none
market	700, 700	200	30/20/25/25	900	none
incident	700, 700	300	45/20/15/20	300	kerb lane of east approach closed 1200 to 2100 s
outage	700, 700	200	30/20/25/25	900	slow loop unreachable 1200 to 2400 s

The measured period is 3600 s after a warm-up of 300 s. Scenario event times are simulation clock times that include the warm-up. Perception is designed to add noise to SUMO ground truth, with a miss probability of 0.05 per object, a relative standard deviation of 0.10 and a minimum standard deviation of 0.5.

Table 12**.** Safety shield and controller parameters

Parameter	Value
Minimum green	10 s
Maximum green	90 s
Starvation limit	180 s
Pedestrian crossing time, east and west arms	16 s
Pedestrian crossing time, south arm	10 s
Amber	3 s
All-red clearance	2 s
Fixed-time greens (phases A, B, C)	35 s, 12 s, 25 s
max_pressure green	15 s
count_timer minimum, step and maximum green	10 s, 5 s, 60 s
Planner horizon, samples, iterations, elites	3, 64, 4, 8
Default subgoal: max wait, validity	120 s, 300 s

Appendix C: Full Results

The table gives, for every scenario and controller, the mean over five evaluation seeds of average waiting time (AWT), average travel time (ATT), average queue length (AQL), throughput and pedestrian waiting time. The `africot_full` controller has no row because it has not been evaluated. Outage rows equal market rows for every controller shown, because those controllers have no slow loop.

Table 13**.** Full results by scenario and controller (mean and standard deviation over five seeds)

Scenario	Controller	AWT (s)	ATT (s)	AQL (veh)	Throughput	Ped. wait (s)	Violations
normal	fixed_time	36.3 (10.1)	100.8 (14.3)	16.8 (4.8)	1698 (26)	19.0 (0.9)	0
normal	count_timer	38.1 (1.3)	104.0 (2.1)	17.8 (0.6)	1704 (17)	13.0 (1.2)	0
normal	max_pressure	17.0 (1.1)	76.5 (1.6)	7.6 (0.6)	1715 (21)	12.5 (0.8)	0
normal	africot_nowm	39.9 (10.2)	104.3 (12.5)	18.2 (4.4)	1702 (15)	19.3 (3.7)	0
normal	africot_noslow	19.3 (1.5)	79.9 (2.3)	8.6 (0.9)	1711 (18)	11.6 (1.1)	0
market	fixed_time	42.4 (11.1)	109.4 (15.5)	18.3 (4.8)	1597 (28)	20.3 (0.4)	0
market	count_timer	44.1 (4.3)	111.5 (5.7)	19.2 (2.0)	1601 (20)	14.1 (0.3)	0
market	max_pressure	25.9 (13.5)	88.7 (18.0)	10.7 (5.6)	1598 (31)	15.5 (5.4)	0
market	africot_nowm	48.2 (9.1)	115.9 (12.0)	20.8 (4.2)	1590 (27)	19.4 (1.6)	0
market	africot_noslow	23.8 (4.4)	86.2 (6.5)	9.8 (1.9)	1601 (24)	12.0 (0.8)	0
incident	fixed_time	56.8 (6.8)	132.9 (9.7)	23.0 (3.9)	1697 (20)	18.9 (0.9)	0
incident	count_timer	52.0 (7.4)	125.6 (10.0)	23.7 (2.7)	1702 (14)	16.5 (0.8)	0
incident	max_pressure	35.8 (8.6)	108.3 (10.9)	15.7 (4.1)	1713 (23)	13.8 (1.0)	0
incident	africot_nowm	72.6 (29.1)	147.5 (33.1)	28.5 (6.1)	1690 (10)	22.2 (3.6)	0
incident	africot_noslow	39.9 (9.2)	115.3 (14.1)	16.9 (2.9)	1708 (18)	14.0 (0.9)	0
outage	fixed_time	42.4 (11.1)	109.4 (15.5)	18.3 (4.8)	1597 (28)	20.3 (0.4)	0
outage	count_timer	44.1 (4.3)	111.5 (5.7)	19.2 (2.0)	1601 (20)	14.1 (0.3)	0
outage	max_pressure	25.9 (13.5)	88.7 (18.0)	10.7 (5.6)	1598 (31)	15.5 (5.4)	0
outage	africot_nowm	48.2 (9.1)	115.9 (12.0)	20.8 (4.2)	1590 (27)	19.4 (1.6)	0
outage	africot_noslow	23.8 (4.4)	86.2 (6.5)	9.8 (1.9)	1601 (24)	12.0 (0.8)	0

Appendix D: Deviations from the Specification

The table summarises the entries of the deviation record kept during the build. Dates are those of the record, and all fall on 6 October 2026.

Table 14**.** Deviations from the build specification

Stage	Deviation	Reason
S1	The first incident was a broken-down vehicle on the kerb lane. It was superseded the same day by the lane closure below.	Traffic passed the vehicle and used the kerb lane downstream, so the lane was not blocked.
S1	Incident: the kerb lane of the east approach is closed to passenger, bus and motorcycle demand with a TraCI permission change from 1200 s to 2100 s, then restored. Vehicles already on the lane at 1200 s may clear it.	This blocks the whole lane up to the stop line, so east-approach traffic shares the median lane.
S1	Event times (incident 1200 to 2100 s, outage 1200 to 2400 s) are read as simulation clock times, including the 300 s warm-up.	The specification does not say which clock; both windows fall inside the measured period either way.
S1	The zero-teleport check counts only teleports with reason jam. Two teleports with reason yield occurred in the tuning runs (market and outage, seed 103) and none with reason jam.	A yield teleport is a junction-blocking artefact of the sublane model with dense pedestrian flow, not a congestion jam.
S1	Market and outage side-road demand is 200 vehicles per hour instead of 550. Other market values are as specified, and normal keeps 300.	At 550 the south arm gridlocked under fixed-time. Of the values tried (550, 300, 275, 250, 225, 200) only 200 met the criterion on the tuning seeds. The side road therefore carries less in market than in normal.
S1 and S2	ATT and AWT include insertion delay and are right-censored at 3900 s for vehicles still in the network or never inserted. Completed-trip means are kept separately.	Without censoring, delay of vehicles queued outside the network vanished, so a controller that starved the side road looked better.
S2	Perception also returns halting vehicles per movement, taken from each vehicle's route, with the same noise as other counts.	Max-pressure needs per-movement queues to distinguish its phases.
S3	The shield enforces the pedestrian crossing time on every phase that gives green to a crosswalk, not only when pedestrians are observed waiting.	Perception can miss a waiting pedestrian, so the unconditional minimum is stricter than specified and never less safe.
S3	The baseline results were rerun through the shield and overwritten, so all reported runs pass through it.	The shield is meant to apply to every controller and cannot be bypassed.
S4	World-model data cover normal, market and incident, not outage.	Outage differs from market only in the slow-loop connectivity

Stage	Deviation	Reason
		window, so the data controllers would produce exact copies of market episodes.
S4	World-model inputs and targets both come from noisy perception, and the persistence baseline uses the noisy current queue.	The model may only see what the deployed controller sees, so errors are measured against noisy observations for both.

Two further points are recorded in the stage notes and the status file rather than the deviation table. The danfo term in the planner cost is approximated from the current danfo share, and the third-gate reproduction used torch 2.12.1 (CPU) because the pinned version was not installable on that machine.